Endpoint Security: Hands-On Laboratory Guide

A 32-Lab Companion to the Endpoint Security Video Course

Joshua Moses | Cybertrace Foundation

Hands-On Laboratory Guide

Contents

About This Guide

This is the lab manual for the Endpoint Security video course. The course teaches a single idea thirty-two different ways: the endpoint is the place where the user, the data, and the attacker all meet on the same machine, and defending it is a layered, ongoing discipline rather than a product you install. Watching the videos gives you the model. This guide is where you build it with your own hands.

Every video in the course ends with a hands-on exercise. This manual turns each of those exercises into a full lab you can actually run, grade, and come back to. There is one lab per video, thirty-two in all, and they are meant to be done in order. The course is built as a progression, harden first, then detect what hardening missed, then manage it all at scale, and the labs follow the same spine. A lab late in the book assumes the endpoint you hardened in Lab 6 and the telemetry you stood up in Lab 3 are still there.

The labs are adversarial on purpose. You will not only configure a control, you will attack it, watch it hold or fail, and decide what would have caught the bypass. That is the whole method of the course compressed into a work habit. A control you have never tried to defeat is a control you do not understand.

Who this is for

The guide assumes you are comfortable installing an operating system, using a command line on both Windows and Linux, and finding your way around system settings. It does not assume you have ever built a security lab, run an attack tool, or read a piece of telemetry. Where a lab needs a concept the course covered in a video, the lab names the video so you can go back. Where a lab needs a skill from another discipline, ethical hacking, forensics, networking, it gives you enough to complete the exercise without sending you off to another course first.

How each lab is built

Every lab in this guide has the same parts, in the same order, so you always know where to look:

- **Lab objectives.** What you will be able to do when the lab is finished, stated as outcomes, not topics.
- **Prerequisites.** The lab state, prior labs, and tools you need in place before you start. Many labs build on an earlier one; this is where that dependency is named.
- **Scenario.** A short framing that puts the exercise in a real defensive situation, so the steps have a reason.
- **Step-by-step instructions.** The actual procedure. Commands are given for the relevant operating system. Where a step would differ between Windows and Linux, both are shown.
- **Lab worksheet.** The record you fill in as you work. The worksheet is the deliverable. A lab you did but did not record is a lab you cannot defend later, and recording what you saw is itself a security habit worth building.
- **Expected outcomes.** What you should observe if the lab worked. Use this to check yourself, not to skip the work.
- **Discussion questions.** Questions that push past the procedure into the judgment the course is really teaching. These are where the learning sticks.
- **Grading rubric.** A point breakdown, for an instructor grading the lab or for you grading yourself honestly.

A note on the course structure

The course, and therefore this guide, moves through five stages:

1. **Foundations (Labs 1 to 5).** What an endpoint is, how it works, the lab you will use for everything that follows, the mindset, and the threats you are defending against.
2. **Hardening and prevention (Labs 6 to 12).** Reducing the attack surface and stopping what you can before it runs.

3. **Detection and response (Labs 13 to 18).** Catching what prevention missed, investigating it, and understanding how controls get bypassed.
4. **Managing endpoints at scale (Labs 19 to 24).** Moving from one machine to a fleet, where endpoint security becomes an operational discipline.
5. **Operations, program, and the road ahead (Labs 25 to 32).** The human factor, the security operation, the program that holds it all together, and where to go next.

Work through them in order and by the end you will have taken a bare machine, hardened it, defended it, broken those defenses, caught the break, managed a small fleet of machines the same way, and assembled the whole thing into a defense you can walk an attack through layer by layer.

Safety, Ethics, and the Rules of This Lab

Read this section before you build anything. Everything in this guide involves attacking machines so you understand how to defend them. That is legitimate and necessary, and it is also exactly the activity that is illegal when done to machines you do not own. The line between a security professional and a criminal is not the technique. It is permission and containment.

Rule 1: You attack only what you own and have isolated. Every attack technique in this guide is run against lab machines you built, inside a lab network that cannot reach anything real. You never point a tool from this guide at a production system, a machine you do not own, or anything on your home or work network that is not part of the lab. "I was just learning" is not a defense anyone has to accept.

Rule 2: The lab is isolated, and you keep it that way. The attack host and the deliberately vulnerable machines in this lab must not touch your real network, the internet unsupervised, or any device that matters. You will build this isolation in Lab 3 and you will confirm it before you run your first attack. If you are ever unsure whether the lab is isolated, stop and check before you continue.

Rule 3: You never test live malware on anything that matters. When a lab uses malware, it uses safe, standardized test files or representative techniques you build yourself, never a real sample pulled from the internet. If you ever handle a real sample for study, it happens on a disposable, isolated machine you are prepared to destroy, and never on your lab's management or host system.

Rule 4: Snapshot before you break, restore after you learn. The virtualization discipline is what makes this safe. Before you compromise a machine, you take a snapshot. After you have investigated it and recorded what you needed, you restore. You never leave a knowingly compromised machine running on your lab network.

Rule 5: Keep the keys out of the blast radius. The credentials, recovery keys, and management access for your lab do not live on the machines you are attacking. A recovery key stored on the encrypted machine it is meant to recover is not protecting anything. This principle, keeping what protects the system separate from the system, runs through the whole course.

Rule 6: What you learn here defends. It does not get deployed against people. The evasion techniques, the credential theft, the deception, all of it exists in this guide so you can build defenses that account for it. Use it that way. If a technique in this guide would be useful to you only for attacking someone, you have misread the purpose of the guide.

If you cannot meet these rules, do not run the attack portions of the labs. The defensive configuration portions are still worth your time, and they are where most of the lasting value is anyway.

The Shared Lab Environment

Lab 3 walks you through building this environment step by step. This section is the reference you come back to. It describes what you are building and why each piece exists, so that when a later lab says "on your Windows defender endpoint," you know exactly which machine it means.

You do not need expensive hardware. The whole lab runs as virtual machines on a single reasonably equipped host. The virtualization lab guide for this course series covers choosing and installing a hypervisor in depth; any of VMware Workstation, VirtualBox, VMware Fusion, or UTM will run this lab. What matters is RAM and disk. Budget at least 16 GB of RAM on the host, 32 GB if you can, and 250 GB of free disk, because you will be keeping snapshots.

The machines

The host. Your real computer. The hypervisor runs here. Nothing in this lab attacks the host, and no lab credential or recovery key is stored anywhere an attacked machine could reach. Treat the host as the one thing in this exercise that stays clean.

WIN-ENDPOINT. A Windows 10 or 11 virtual machine, standing in for a normal user's workstation. This is your primary endpoint to defend. Most hardening, prevention, and detection labs land here first. Use a Windows evaluation image so you are not burning a license on a machine you will wipe repeatedly.

WIN-SERVER. A Windows Server evaluation VM, standing in for a server endpoint and, later, for the small-scale management and directory services the fleet labs need. In a home lab this one machine plays several roles. That is fine and realistic for a small environment.

LIN-ENDPOINT. A Linux VM, Ubuntu Server or Fedora, standing in for a Linux endpoint. Several labs defend this machine; a few use it as a second endpoint so you can see a control applied across different operating systems.

ATTACK-HOST. The attacker's machine. This is where you run the attack tools that test your defenses. Keep it clearly separate in your mind and on your network from the machines you are defending. It should be able to reach the endpoints across the lab network and nothing else. You do not need a heavyweight penetration-testing distribution for this guide; a clean Linux VM with a handful of named tools installed as each lab calls for them is cleaner, easier to reason about, and does not drown you in software you will never use.

MONITOR. A machine that collects the logs and events your endpoints generate. In the early labs this can be a role on WIN-SERVER or a small Linux VM running a log collector. By the fleet and operations labs it becomes the central point your telemetry flows into. Standing this up early, even in a minimal form, means every later lab has somewhere to send its signal.

The network

All of these machines sit on an isolated, host-only or internal virtual network. They can talk to each other. They cannot talk to your real network or to the internet except when a specific lab deliberately and temporarily allows it, under your supervision, for something like fetching a benchmark or an update. The default state of this lab is sealed.

Draw this network once, by hand, early. Knowing which machine can reach which other machine is not a diagram nicety. It is the thing that keeps your attack exercises legal and safe, and it is the first thing you will check when something behaves unexpectedly.

The disciplines you keep throughout

- **Snapshot before every compromise.** Name the snapshot for what the machine is about to go through ("pre-ransomware-lab-13"), so you can find it again.
- **Restore when you are done learning from a broken machine.** Do not accumulate compromised VMs.
- **Record as you go.** The worksheet in each lab is the record. Keep the finished worksheets together; several late labs ask you to compare against an earlier one.
- **Keep the attacker's machine and the defender's machines distinct.** Never install a defensive agent on the attack host or an attack tool on a machine you are trying to defend, unless a lab explicitly tells you to, for a reason.

Part I: Foundations

Lab 1: Inventory and Classify Your Endpoints

Companion to Video 1: What Endpoint Security Actually Is

Lab objectives

By the end of this lab you will be able to:

- Identify every endpoint in a home or small environment, including the ones that do not look like computers.
- Classify each endpoint by its type, its exposure, and the sensitivity of the data it holds.
- Explain why the same device can carry very different risk depending on exposure and data, and defend a ranking of which endpoints matter most.

Prerequisites

- No lab machines are required yet. This lab is done against your real home or small-office environment, as an inventory exercise only. You will look at and catalog devices. You will not change or attack anything.
- A notebook or spreadsheet for the inventory.

Scenario

You have been asked to secure an environment, and the first honest answer to "what are we protecting?" is almost always "we are not entirely sure." You cannot defend endpoints you do not know you have. Before any control, before any tool, there is a list. This lab builds that list and, more importantly, builds the habit of seeing endpoints everywhere, because the device nobody counted is the device nobody secured.

Step-by-step instructions

1. **List the obvious endpoints.** Walk through your space and write down every laptop, desktop, phone, and tablet. For each, note who uses it and what operating system it runs.
2. **Find the servers and always-on machines.** Anything that runs continuously and serves something to other devices counts. A home NAS, a media server, a small business file server, a machine running a database. Add each one.
3. **Find the non-traditional endpoints.** This is the part most people miss. A smart TV, a streaming stick, a networked printer, a security camera, a smart thermostat, a voice assistant, a game console, a connected doorbell. Each of these is a computer that connects to your network and runs code. Add every one you can find. Check your router's list of connected devices to catch the ones you forgot.
4. **Classify each endpoint by type.** Group your list: user workstation, mobile device, server, networked infrastructure, Internet of Things or embedded device. The type tells you roughly how much control you will have over it later. You can harden a laptop deeply. You often cannot harden a smart bulb at all.
5. **Classify each endpoint by exposure.** For each device, ask: can it be reached from the internet, or only from inside the network? Does it travel outside the network, like a laptop or phone? Is it physically accessible to people who are not you? Rate each as low, medium, or high exposure, and write down the reason.
6. **Classify each endpoint by data.** For each device, ask what data lives on it or passes through it. A laptop with financial records holds more risk than an identical laptop used only for streaming. Rate the data sensitivity low, medium, or high.
7. **Rank the endpoints.** Using type, exposure, and data together, order your list from the endpoint you would most need to protect to the one you would least need to protect. There is no single correct order. What matters is that you can defend yours.

Lab worksheet

Fill in one row per endpoint.

#	Device name	Type	Operating system	Exposure (L/M/H)	Exposure reason	Data sensitivity (L/M/H)	Data reason	Overall priority rank
1								
2								
3								

Below the table, answer in writing:

- How many endpoints did you find in total?
- How many were non-traditional devices you would not normally think of as computers?
- Which device surprised you most, and why?

Expected outcomes

A completed inventory that is longer than you expected. Most people find two to three times more connected devices than they would have guessed before counting. You should finish the lab with a clear sense that "endpoint" is a much wider category than "laptop," and with a defensible ranking of what matters most in your own environment.

Discussion questions

1. Two identical laptops can carry completely different risk. What makes the difference, and which of your three classification axes captured it?
2. You almost certainly found at least one device you cannot meaningfully secure, like a cheap IoT gadget. If you cannot harden it, what else could you do to limit the risk it creates? (The course returns to this in Lab 24.)
3. An attacker does not care about your most important endpoint. They care about your weakest reachable one. How does that change the way you read your own ranked list?

Grading rubric

Criterion	Points
Complete inventory including non-traditional and IoT endpoints	25
Each endpoint correctly classified by type	15
Exposure rated with a stated reason for each device	20
Data sensitivity rated with a stated reason for each device	20
Defensible overall priority ranking	10
Written reflection answered thoughtfully	10
Total	**100**

Lab 2: Map the Attack Surface of an Endpoint

Companion to Video 2: How Endpoints Work and Where They Are Attacked

Lab objectives

By the end of this lab you will be able to:

- Trace what happens on an endpoint from power-on through to running processes and network connections.
- Identify the five attack surfaces of an endpoint, the operating system, the applications, the users, the network, and the physical device, on a real machine.
- Explain, for each surface, where an attacker would try to get in, so that later hardening labs have something concrete to defend.

Prerequisites

- One endpoint you own that you can inspect freely. This can be your WIN-ENDPOINT lab VM if you have already built it, or your own workstation. You will only observe, not change anything.
- Administrative or standard access to run basic inspection commands.

Scenario

You cannot defend a machine whose internals you do not understand. The difference between someone who installs a security agent and someone who actually defends an endpoint is that the second person knows what the agent is watching and why. This lab takes one machine apart conceptually, not physically, so that every control in the rest of the course has a place to attach.

Step-by-step instructions

1. **Observe the boot process.** Restart the machine and watch what happens before the operating system loads. Note whether it uses UEFI, whether Secure Boot is on or off (visible in firmware settings), and how little stands between power-on and code running with the highest privilege. You are not changing anything, only seeing how much power lives in the earliest code. (Lab 11 defends this.)
2. **Look at the privilege model.** Log in and determine whether your account is a standard user or an administrator.
   - Windows: open a command prompt and run `whoami /groups` and look for the Administrators group and the integrity level.
   - Linux: run `id` and note whether you are in sudo or wheel, and run `whoami`.

Write down what your account can and cannot do without elevation.

1. **List running processes.** See what is actually executing.
   - Windows: open Task Manager, then run `tasklist` in a command prompt for the full list.
   - Linux: run `ps aux`.

Pick five processes you do not recognize and find out what each one is. Attackers hide among exactly these.

1. **Examine the file system and configuration store.** Look at where the operating system keeps its settings.
   - Windows: open the Registry Editor (`regedit`) and look at, but do not change, the Run keys under `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run`. This is one place attackers set themselves to start automatically. (Lab 14 hunts here.)
   - Linux: look at `/etc/systemd/system/` and the contents of `crontab -l`. Same idea, different store.
1. **Map the network stack.** See how the machine talks to the world.
   - Windows: run `netstat -ano` to see active connections and listening ports.
   - Linux: run `ss -tunap` or `netstat -tunap`.

Identify every listening port and what process owns it. A listening port is a door. (Lab 10 controls these.)

1. **Draw the five surfaces.** On one page, draw the endpoint and label the five attack surfaces: the operating system, the applications installed on it, the user accounts, the network connections, and the physical device itself. Under each, write one concrete thing you observed in the steps above that belongs to that surface.

Lab worksheet

Boot observations: Firmware type (UEFI/BIOS): ____ Secure Boot on/off: ____ One thing that surprised you about how early powerful code runs: ____

Privilege: My account type: ____ Three things it can do without elevation: ____ Three things it cannot: ____

Processes: Five unrecognized processes and what each turned out to be:

Process name	What it is	Legitimate?

Autostart locations checked: ____ Number of autostart entries found: ____

Network: Listening ports and owning process:

Port	Protocol	Owning process	Why is this open?

The five surfaces: Attach your one-page drawing, with one observed item noted under each surface.

Expected outcomes

A single endpoint, fully mapped. You should be able to point at any of the five surfaces and name something real on your machine that lives there. You should also have found at least one listening port or autostart entry you could not immediately explain, which is the normal and instructive result. Not knowing yet is the honest starting point.

Discussion questions

1. Of the five surfaces, which did you find hardest to inspect, and what does that difficulty tell you about how hard it will be to defend?
2. You found processes and ports you could not identify. An attacker relies on exactly that uncertainty. What would it take to get to a state where every process and port on your machine is one you can account for?
3. The boot process runs with the most power and is examined the least by most users. Why do you think the most powerful code gets the least attention, and what does that pattern suggest about where attackers invest?

Grading rubric

Criterion	Points
Boot process observed and firmware/Secure Boot state recorded	15
Privilege model correctly determined and documented	15
Running processes listed and five investigated	20
Autostart/configuration stores located and examined	15
Network connections mapped with owning processes identified	20
Five-surface drawing complete with a real observed item under each	15
Total	**100**

Lab 3: Build Your Endpoint Security Lab

Companion to Video 3: Building Your Endpoint Security Lab

Lab objectives

By the end of this lab you will be able to:

- Stand up a working lab of endpoints to defend and an isolated attack host.
- Confirm that the lab is isolated from your real network and the internet.
- Compromise a lab endpoint from the attack host and observe that compromise from your telemetry, proving the whole loop works before any real exercise depends on it.

Prerequisites

- A host machine with the RAM and disk described in the Shared Lab Environment section.
- A hypervisor installed. The virtualization lab guide for this series covers this in full; any of VMware Workstation, VirtualBox, Fusion, or UTM is fine.
- Evaluation images: Windows 10/11, Windows Server, and a Linux distribution (Ubuntu Server or Fedora).

Scenario

Every later lab assumes this environment exists and works. This is the lab that makes all the others real. You will build the machines, seal the network, and then do a complete end-to-end dry run, attack a machine, see the attack in your logs, restore the machine, so that when a real lab compromises an endpoint in Lab 13, you already trust that your telemetry will show it.

Step-by-step instructions

1. **Create the virtual network.** In your hypervisor, create a host-only or internal network. This is the lab network. It must not be bridged to your real network. Every lab machine attaches here.
2. **Build the endpoints to defend.** Create three VMs: WIN-ENDPOINT (Windows 10/11), WIN-SERVER (Windows Server), and LIN-ENDPOINT (Linux). Install the operating systems, apply current updates once (temporarily allowing internet access under supervision, then removing it), and take a clean baseline snapshot of each named `clean-baseline`.
3. **Build the attack host.** Create a clean Linux VM named ATTACK-HOST on the same lab network. Install only what you need as you go; for this lab you need a basic set of tools to generate a test compromise, which the lab names in step 6. Snapshot it as `clean-attack`.
4. **Build the monitor.** Create a small VM named MONITOR, or designate a role on WIN-SERVER, to collect logs. Install a simple log collector or enable centralized event forwarding. The goal for now is modest: somewhere that endpoint logs can be sent and read in one place.
5. **Confirm isolation. This is the gate. Do not skip it.** From WIN-ENDPOINT and from ATTACK-HOST, try to reach the internet (`ping` a public address, open a browser). Both should fail. Try to reach your real home router or another real device. That should fail too. Then confirm the lab machines can reach each other. If any lab machine can reach the real world or your real network when it should not, fix the network before going further. Record the result of each test.
6. **Set up endpoint telemetry.** On WIN-ENDPOINT, enable detailed logging: turn on process creation auditing (and command-line logging) and PowerShell logging, and confirm events are being written. Point the machine's logs at your MONITOR. On LIN-ENDPOINT, confirm `auditd` or equivalent is running and logging. You now have endpoints that record what happens on them.
7. **Run the end-to-end test.** Snapshot WIN-ENDPOINT as `pre-test-lab3`. From ATTACK-HOST, perform one simple, safe action that should generate a visible event on WIN-ENDPOINT, for example, an authentication attempt against it, or copy and run a harmless test executable that writes a file and makes a network connection back to

ATTACK-HOST. On MONITOR, find the corresponding events. You are proving that an action on the attacker's side shows up in your telemetry.

8. **Restore.** Restore WIN-ENDPOINT from pre-test-lab3. Confirm it is clean. The reset discipline works.

Lab worksheet

Machines built:

VM name	OS	Role	Clean snapshot taken?
WIN-ENDPOINT		Defend	
WIN-SERVER		Defend / services	
LIN-ENDPOINT		Defend	
ATTACK-HOST		Attack	
MONITOR		Telemetry	

Isolation tests:

Test	Expected	Result
WIN-ENDPOINT to internet	Fail	
ATTACK-HOST to internet	Fail	
Any lab machine to real network	Fail	
Lab machines to each other	Succeed	

End-to-end test: Action taken from ATTACK-HOST: ____ Event(s) observed on MONITOR: ____ Time from action to visible event: ____

Restore: Snapshot restored cleanly? ____

Expected outcomes

A sealed lab in which you can attack a machine and watch the attack appear in your logs, then put the machine back. If isolation passed, telemetry showed your test action, and the restore returned the machine to clean, the lab is ready and every later exercise rests on solid ground. If any of the three failed, the fix is worth more than moving on.

Discussion questions

1. Isolation is the one step this lab tells you not to skip. What is the worst realistic outcome of running the later attack labs on a lab that was not actually isolated?
2. Your end-to-end test proved an action became a visible event. What kinds of attacker actions might *not* produce an event with the logging you set up, and how would you ever know? (Lab 13 and Lab 18 return to this.)
3. The snapshot-and-restore discipline makes compromise safe and reversible. How does having that safety net change what you are willing to try in the labs ahead?

Grading rubric

Criterion	Points
All five VMs built and clean snapshots taken	20
Lab network created and not bridged to the real network	15
Isolation confirmed with each test recorded	25
Endpoint telemetry enabled and flowing to the monitor	20

Criterion	Points
End-to-end attack-to-telemetry test succeeded and was documented	15
Snapshot restore confirmed clean	5
Total	**100**

Lab 4: Defend and Bypass a Single Control on Paper

Companion to Video 4: The Endpoint Security Mindset and Method

Lab objectives

By the end of this lab you will be able to:

- Take any single endpoint control and reason through both how it defends and how an attacker would bypass it.
- Decide what additional control or signal would detect the bypass, applying the defense-in-depth principle before you deploy anything.
- Articulate the usability cost of a control and predict whether users would defeat it.

Prerequisites

- No machines required. This is a structured thinking exercise done on paper or in a document. It is deliberately placed before the hands-on hardening labs so that you carry the mindset into them.
- The attack-surface map you made in Lab 2 is useful reference.

Scenario

The course calls this the bypass instinct: the discipline of asking, for every control you deploy, how an attacker would defeat it. A defender who only knows how a control works, and not how it fails, will trust a single control to hold and be surprised when it does not. This lab builds the habit on paper, cheaply, before you build it on machines, expensively.

Step-by-step instructions

1. **Choose a control.** Pick one real endpoint control. Good choices for this lab: requiring a screen lock after inactivity, removing administrator rights from users, blocking USB storage devices, or requiring a password at boot. Pick one.
2. **Write how it defends.** In plain language, state exactly what attack this control stops and how. Be specific. "A screen lock stops someone who walks up to an unattended, logged-in machine from using the existing session." Name the attack it defeats.
3. **Write how an attacker bypasses it.** Now switch sides. Given motivation and access, how would someone defeat this control? Think across the five surfaces from Lab 2. For the screen lock: they could catch the machine before it locks, shoulder-surf the password, use a separate attack that does not need the session at all, or coerce the user. List at least three distinct bypasses.
4. **Rate the usability cost.** How much does this control cost the user in friction? A control users hate is a control users defeat, by propping it open, writing the password down, or demanding it be removed. Rate the friction low, medium, or high and predict how users would react over a month.
5. **Decide what detects the bypass.** For each bypass you listed, write what would produce a signal if it happened. If someone bypassed the screen lock by catching the machine unlocked, what log, alert, or other control would reveal it? If nothing would, say so plainly. "Nothing I currently have would detect this" is the single most valuable sentence a defender can write, because it is the gap.
6. **State the layered conclusion.** In one or two sentences, describe the second control you would add so that this one control is not trusted to hold alone.

Lab worksheet

Control chosen: ____

How it defends (name the specific attack it stops): ____

Three bypasses:

#	How the attacker bypasses it	Which surface (from Lab 2) it uses
1		
2		
3		

Usability cost: Friction (L/M/H): ____ Predicted user reaction over a month: ____

Detection of each bypass:

Bypass #	What would produce a signal?	Do I currently have it?
1		
2		
3		

Layered conclusion: The second control I would add: ____

Expected outcomes

A one-page analysis that treats a single control honestly: what it stops, how it fails, what it costs, and what would catch the failure. You should finish with at least one bypass for which you have no current detection. That gap is not a failure of the lab. It is the point of the lab, and it is the reasoning you will apply to every real control you deploy from here on.

Discussion questions

1. Which was harder to write, the defenses or the bypasses, and what does that say about which instinct you need to develop more?
2. You rated a usability cost. Think of a control you have personally seen someone defeat because it was too annoying. What was the security consequence of that human reaction?
3. The course says every control should either prevent an attack or produce evidence of one. Did your chosen control do both, one, or neither, and what would you change so it does both?

Grading rubric

Criterion	Points
Control chosen and its specific defensive purpose stated	15
At least three distinct, plausible bypasses identified	25
Each bypass mapped to an attack surface	10
Usability cost rated with a realistic predicted user reaction	15
Detection analyzed per bypass, with gaps named honestly	25
Layered conclusion proposes a sound second control	10
Total	**100**

Lab 5: Observe Endpoint Attack Techniques Safely

Companion to Video 5: Understanding Endpoint Threats

Lab objectives

By the end of this lab you will be able to:

- Execute a set of safe, representative endpoint attack techniques in your lab.
- Observe exactly what each technique does to an endpoint, on disk, in memory, in the registry or configuration store, and on the network.
- Map each technique to the category of threat it represents, building the reference you will use to test every control in the rest of the course.

Prerequisites

- The lab from Lab 3, built and isolated.
- WIN-ENDPOINT and ATTACK-HOST running, with telemetry flowing to MONITOR.
- A fresh `clean-baseline` snapshot of WIN-ENDPOINT.

Scenario

Every control in the rest of this course counters a specific attacker technique. If you do not know what the techniques actually do, you are configuring controls on faith. This lab has you run a small, safe, representative set of techniques against your own lab endpoint and watch precisely what each one does, so that when Lab 6 claims a hardening setting "blocks" something, you know exactly what it is blocking.

Everything here uses safe test files and techniques you build yourself. Nothing uses a real malware sample.

Step-by-step instructions

Snapshot WIN-ENDPOINT as `pre-threats-lab5` before you begin. Restore it at the end.

1. **A safe malware stand-in.** Create the standard EICAR antivirus test file on WIN-ENDPOINT (a harmless text string every antivirus recognizes as a test). Observe whether anything reacts. This stands in for commodity malware delivered to disk. Note what detection, if any, fires. (You will deploy real antivirus against this in Lab 12.)
2. **A delivery mechanism.** From ATTACK-HOST, host a harmless test file on a simple web server and download it to WIN-ENDPOINT through a browser, then through a command-line download. Observe the difference in what each leaves behind. This represents the download delivery vector.
3. **A persistence mechanism.** On WIN-ENDPOINT, add a harmless command (something that writes a timestamp to a file) to a Run registry key and to a scheduled task. Reboot. Confirm it ran on startup. This is persistence, the attacker ensuring they survive a reboot. Record exactly where you placed it. (Lab 14 hunts for exactly this.)
4. **A living-off-the-land action.** Using only tools already built into Windows (for example, a PowerShell one-liner that downloads and runs a harmless script from ATTACK-HOST entirely in memory), perform an action without dropping a new executable on disk. Observe how little lands on disk compared to step 2. This is the fileless technique that defeats controls looking for foreign files. (Lab 18 returns to this as evasion.)
5. **A post-compromise action.** Simulate what an attacker does after landing: run built-in commands that enumerate the system, `whoami`, `net user`, `systeminfo`, `ipconfig /all`, `netstat`. These are the first things an intruder runs to understand where they are. Watch them generate process-creation events in your telemetry.
6. **Observe each in your telemetry.** For every technique above, go to MONITOR and find what it produced. Build the mapping: technique on one side, what it left on disk / in memory / in the registry / on the network / in the logs on the other.
7. **Restore.** Restore WIN-ENDPOINT from `pre-threats-lab5`.

Lab worksheet

For each technique, record what you observed.

Technique	Threat category	On disk	In memory	In registry/config	On network	Visible in telemetry?
EICAR test file	Commodity malware					
Browser + CLI download	Delivery					
Run key + scheduled task	Persistence					
In-memory PowerShell	Fileless / LOL					
Enumeration commands	Post-compromise					

Below the table:

- Which technique left the least evidence on disk? ____
- Which technique was hardest to see in your telemetry? ____
- Which single data source (disk, memory, registry, network, logs) revealed the most across all five techniques? ____

Expected outcomes

A reference table you will use for the rest of the course. The fileless, in-memory technique should leave markedly less on disk than the download, and may be the hardest to see, which is exactly why later detection labs exist. You should finish understanding, concretely rather than abstractly, that different attacks live in different places, and that no single data source sees all of them.

Discussion questions

1. The persistence you planted survived a reboot. If you did not already know where you had put it, how would you find it? Hold that thought; Lab 14 is the answer.
2. The in-memory technique left little on disk. What does that tell you about the limits of any control that works by inspecting files?
3. You mapped each technique to where it leaves evidence. Looking at your table, which attack surface from Lab 2 is involved in the most techniques, and does that change your sense of where to invest in defense first?

Grading rubric

Criterion	Points
All five techniques executed safely in the isolated lab	25
Behavior of each recorded across disk, memory, registry, network	25
Telemetry checked and visibility noted for each technique	20
Correct mapping of each technique to its threat category	15
Reflection questions answered using observed evidence	10
Snapshot restore confirmed	5
Total	**100**

Part II: Hardening and Prevention

Lab 6: Apply a Hardening Benchmark and Measure What It Blocks

Companion to Video 6: Operating System Hardening

Lab objectives

By the end of this lab you will be able to:

- Apply a published hardening benchmark to a lab endpoint from a secure baseline.
- Confirm the hardened endpoint still works for a normal user, the test that separates real hardening from hardening that gets rolled back.
- Measure which of the attack techniques from Lab 5 the hardening now blocks, and which it does not.

Prerequisites

- WIN-ENDPOINT from a clean-baseline snapshot.
- Your completed Lab 5 worksheet, the technique reference.
- Access to a published benchmark. The CIS Benchmarks and the Microsoft security baselines are the standard starting points; either is fine. Download the relevant benchmark document or tooling to your host first.

Scenario

Hardening comes first because the most effective endpoint security is the attack surface you removed before an attacker could reach it. But hardening has two failure modes the course names directly: the baseline that broke the users' work and got rolled back, and the setting that looked secure and changed nothing. This lab guards against both by requiring you to prove the machine still works and to measure what actually changed.

Step-by-step instructions

1. **Snapshot and record the starting state.** Snapshot WIN-ENDPOINT as pre-hardening-lab6. First, re-run two or three of the Lab 5 techniques against the unhardened machine and confirm they still behave as recorded. This is your before measurement.
2. **Establish the secure baseline source.** Open your chosen benchmark (CIS or Microsoft baseline) for your Windows version. Read the first section so you understand that these settings are a proven, published starting point rather than someone's opinion.
3. **Remove what is not needed.** Before applying settings, reduce the surface: review installed applications and remove anything the machine does not need, and review running services and disable ones that are not required. Record what you removed. Absence closes attack paths.
4. **Apply the benchmark settings.** Apply a defined subset of the benchmark, for a home lab, applying the full benchmark at once tends to break things in ways that are hard to diagnose, so apply one section at a time (account policies, then audit policy, then the security options). You can apply Microsoft baselines via their provided group policy objects, or apply CIS settings manually or with their tooling. Snapshot between major sections so you can roll back a single section if it breaks something.
5. **Confirm the user can still work.** This is the step people skip and regret. Log in as a standard user and do normal things: open applications, browse (if the lab allows it temporarily), save files, print to a lab printer if you have one, run the software a real user would. Anything broken by hardening gets recorded. A benchmark setting that breaks legitimate work will be rolled back in the real world, so you need to know now.
6. **Enable patch currency as part of the baseline.** Confirm the machine is fully patched. An unpatched endpoint is hardened in name only. (Lab 20 handles patching at scale; here, just confirm the single machine is current.)

7. **Measure what the hardening blocks.** Re-run every Lab 5 technique against the hardened machine. For each, record whether it still works, is blocked, or is partially blocked. This is your after measurement, and the difference between before and after is the actual value of the hardening.

Lab worksheet

Surface reduction: Applications removed: ____ Services disabled: ____

Benchmark applied: Source (CIS / Microsoft baseline): ____ Sections applied: ____

Usability check:

Normal user task	Worked after hardening?	If broken, which setting?
Open common applications		
Save and open files		
Print		
Run required software		

Before and after, per technique:

Lab 5 technique	Before (worked?)	After hardening	Blocked / partial / not blocked
EICAR test file			
Download delivery			
Persistence (Run key / task)			
In-memory PowerShell			
Enumeration commands			

Net result: Techniques fully blocked by hardening alone: ____ / 5

Expected outcomes

A measurably more secure endpoint that a user can still work on. Hardening typically blocks or constrains some techniques, often the clumsier delivery and persistence methods, while leaving others, particularly the living-off-the-land and enumeration techniques that use legitimate built-in tools, largely untouched. That gap is expected and important: it is the reason hardening is the foundation and not the whole house. You should also have found at least one setting that broke something and had to be reconsidered.

Discussion questions

1. Which techniques survived hardening, and what do they have in common? What does that tell you about the limits of attack-surface reduction as a strategy?
2. You almost certainly hit a setting that broke legitimate work. In a real organization, who decides whether that setting stays, and what information would they need from you to decide well?
3. The course warns about "the setting that looked secure and changed nothing." Did you apply any setting whose before-and-after made no measurable difference to your five techniques? How would you tell a cosmetic setting from a real one?

Grading rubric

Criterion	Points
Surface reduction performed and documented	15

Criterion	Points
Benchmark applied methodically with snapshots between sections	20
Usability confirmed for normal user tasks	20
Patch currency confirmed	5
Before-and-after measurement completed for all five techniques	30
Net result tallied and reflection answered	10
Total	**100**

Lab 7: Remove Admin Rights and Control Local Credentials

Companion to Video 7: Privilege, Accounts, and Local Access Control

Lab objectives

By the end of this lab you will be able to:

- Remove administrative rights from a standard user and confirm which attacks that defeats.
- Randomize and rotate local administrator passwords so the same credential does not unlock every machine.
- Demonstrate the lateral-movement technique that shared local credentials enable, and confirm your fix defeats it.

Prerequisites

- WIN-ENDPOINT and WIN-SERVER from clean baselines, both on the lab network.
- ATTACK-HOST running.
- Snapshots of both Windows machines before you start (pre-privilege-lab7).

Scenario

The difference between a contained compromise and a total one is often local administrative rights. Two problems dominate this lab: users running as administrators, and the same local administrator password set on every machine. The second is what lets an attacker who compromises one machine walk to all the others. This lab fixes both and proves the fix.

Step-by-step instructions

1. **Set up the vulnerable condition.** On WIN-ENDPOINT and WIN-SERVER, set the same local administrator password on both (this is the common, dangerous real-world state). Create a normal user account on WIN-ENDPOINT and make it a member of the local Administrators group. Snapshot.
2. **Demonstrate the shared-credential problem.** From ATTACK-HOST, having obtained the local admin password hash or password from WIN-ENDPOINT (in the lab, you can simply use the password you set), authenticate to WIN-SERVER using that same local admin credential. Confirm it works. This is lateral movement: one credential, many machines. Record it.
3. **Demonstrate the admin-user problem.** As the admin-rights user on WIN-ENDPOINT, run one of the persistence or configuration-change techniques from Lab 5 that requires elevation, and confirm it succeeds without any prompt or barrier. The user's own admin rights handed it over.
4. **Remove admin rights.** Change the normal user on WIN-ENDPOINT to a standard user, removed from the Administrators group. Log in as that user.
5. **Confirm what removing admin defeats.** Re-run the elevation-requiring technique from step 3 as the now-standard user. Confirm it is blocked or forces an elevation prompt the standard user cannot satisfy. Record which Lab 5 techniques this change defeats.
6. **Eliminate the shared local credential.** Set a unique, randomized local administrator password on each machine. The real-world tool for this at scale is Windows LAPS (Local Administrator Password Solution), which randomizes and rotates each machine's local admin password and stores it centrally; configure LAPS if you can, or simulate it by setting distinct strong passwords and documenting a rotation schedule. Snapshot after.
7. **Confirm the lateral-movement fix.** Repeat step 2: from ATTACK-HOST, try the WIN-ENDPOINT local admin credential against WIN-SERVER. It should now fail, because the credential is no longer shared. The single compromise no longer unlocks the second machine.
8. **Protect credentials in memory.** Confirm that credential-protection features (such as Credential Guard on supported Windows versions) are enabled where available, reducing what an attacker can harvest from a running machine. (Lab 18 returns to credential theft as an evasion concern.)

Lab worksheet

Step	Before fix	After fix
Shared local admin credential reaches second machine?		
Elevation-requiring technique succeeds for the user?		
Credentials harvestable from memory?		

Techniques defeated by removing admin rights (from Lab 5): ____

Local admin password approach: LAPS configured / simulated with unique passwords + rotation schedule: ____

Lateral movement after fix: Succeeded / Failed: ____

Expected outcomes

Two concrete wins you can demonstrate. After removing admin rights, elevation-requiring attacks fail for the normal user. After eliminating the shared local credential, a compromise of one machine no longer authenticates to the other. You should be able to show both the broken "before" and the fixed "after," which is the evidence that these two unglamorous controls do more to contain an intrusion than most expensive tools.

Discussion questions

1. Removing admin rights is one of the highest-impact endpoint controls and one of the most resisted by users. Why is it resisted, and what would you do to make a standard-user environment workable for people who genuinely need to install things?
2. The shared local admin password is almost universal in environments that have not addressed it. Why is it so common, and why is it so dangerous specifically for lateral movement rather than initial access?
3. After your fixes, an attacker who lands on WIN-ENDPOINT is more contained but not powerless. What can they still do with only standard-user rights on one isolated machine?

Grading rubric

Criterion	Points
Vulnerable shared-credential and admin-user conditions set up correctly	15
Lateral movement demonstrated before the fix	20
Admin rights removed and the defeated techniques confirmed	20
Unique/randomized local admin passwords implemented (LAPS or simulated)	20
Lateral movement shown to fail after the fix	20
Credential-in-memory protection addressed	5
Total	**100**

Lab 8: Configure Application Control and Then Bypass It

Companion to Video 8: Application Control and Execution Prevention

Lab objectives

By the end of this lab you will be able to:

- Configure application control on a lab endpoint so that only approved code runs.
- Confirm it blocks a malware stand-in that hardening and antivirus might miss.
- Find a bypass through a legitimately approved application, understanding first-hand why application control is powerful but not absolute.

Prerequisites

- WIN-ENDPOINT from the hardened state you reached in Lab 6, or a clean baseline if you prefer to start fresh.
- Snapshot pre-appcontrol-lab8.
- The EICAR test file and the in-memory PowerShell technique from Lab 5.

Scenario

An endpoint that only runs approved code defeats most malware outright, because most malware is, by definition, code you never approved. This inverts the antivirus model: instead of trying to block everything bad, you permit only what is known good. It is one of the strongest controls available, and this lab has you both deploy it and defeat it, because the bypass through an approved application is the thing you most need to understand before you rely on it.

Step-by-step instructions

1. **Snapshot.** Snapshot WIN-ENDPOINT as pre-appcontrol-lab8.
2. **Choose the mechanism.** Windows offers AppLocker and Windows Defender Application Control (WDAC). AppLocker is easier to configure for a first lab; use it. Plan to allowlist the applications a normal user needs and block everything else from running.
3. **Build the allowlist.** Configure AppLocker to allow executables in the standard protected locations (Program Files, Windows) and to block execution from locations a user can write to (their profile, Downloads, Temp). This single pattern, no execution from user-writable locations, stops a large share of commodity malware, which typically lands and runs from exactly those folders. Enforce the rules.
4. **Confirm it blocks the obvious.** Copy the EICAR test file (renamed to an executable extension for this test) or a harmless test executable into the user's Downloads folder and try to run it. It should be blocked by application control before any antivirus even weighs in. Record the block.
5. **Add script and macro control.** Constrain the scripting features attackers abuse: set PowerShell to Constrained Language Mode where possible, and configure Office macro settings to block macros from the internet. These address the fileless techniques from Lab 5.
6. **Confirm the usability cost.** Try to install or run a legitimate application from a user-writable location. It is blocked too. This is the honest cost: strict application control fights against users installing what they need. Record what broke for the user.
7. **Now bypass it.** Here is the lesson. Application control that allows approved applications can often be defeated by making an approved application do the attacker's work. Using only an allowed, signed, built-in Windows tool (the class of technique known as living-off-the-land binaries, or LOLBins), execute code without dropping a new blocked executable. For example, use an approved scripting host or system utility that is on your allowlist to run your in-memory PowerShell action from Lab 5. Confirm it runs despite application control. Record exactly which approved tool you used.
8. **Record the gap and restore.** Note that the bypass succeeded through something your allowlist explicitly permitted. This is why application control is a layer, not a wall. Restore from snapshot.

Lab worksheet

Allowlist configured: Allowed locations: ____ Blocked locations: ____ Enforcement on? ____

Blocking confirmed:

Test	Blocked?
Test executable in Downloads	
Legitimate app from user folder (usability cost)	

Script/macro control: PowerShell mode set: ____ Macro policy set: ____

The bypass: Approved tool used to bypass: ____ Did code run despite application control? ____

Expected outcomes

Application control that visibly stops malware dropped into user folders, visibly costs the user some freedom, and is visibly bypassable through an approved tool. All three are the point. You should leave this lab convinced that application control is one of the most powerful single controls you can deploy and equally convinced that it cannot be the only one, because you personally walked around it using something it was configured to trust.

Discussion questions

1. The "no execution from user-writable locations" rule blocks a large fraction of commodity malware with one idea. Why does so much malware run from those locations, and what kind of attacker is unaffected by this rule?
2. Your bypass used an approved, trusted tool. How would you detect misuse of a tool that is supposed to be there? (This is a preview of Lab 13 and Lab 14.)
3. Strict application control has a real usability cost that causes real revolts. How would you phase it in across a group of users so that it sticks rather than getting rolled back?

Grading rubric

Criterion	Points
Application control configured with a sound allowlist pattern	25
Malware stand-in confirmed blocked	15
Script and macro control added	15
Usability cost observed and recorded	10
Bypass through an approved tool achieved and documented	25
Reflection connects the bypass to the need for layering	10
Total	**100**

Lab 9: Encrypt the Disk and Find the Limits of Encryption

Companion to Video 9: Data Protection and Encryption on the Endpoint

Lab objectives

By the end of this lab you will be able to:

- Enable full-disk encryption on a lab endpoint and confirm the data is unreadable when the device is off.
- Demonstrate why full-disk encryption does not protect a running, unlocked device.
- Reason about where the encryption key lives and how it is protected and recovered.

Prerequisites

- WIN-ENDPOINT (BitLocker) or LIN-ENDPOINT (LUKS) from a clean baseline. The lab works on either; the worksheet covers both.
- A way to mount the VM's virtual disk from another machine or the host, to simulate an attacker reading the disk offline. On a lab VM this means attaching the virtual disk to another VM as a secondary disk.
- Snapshot before starting.

Scenario

A laptop that is lost or stolen is a breach, unless the data on it was protected. Full-disk encryption is the control that turns a stolen device into a stolen brick. But it protects exactly one situation, the device at rest and powered off, and the most common real failure is trusting it to protect situations it never covered. This lab makes both the protection and its boundary concrete.

Step-by-step instructions

1. **Snapshot** as pre-encryption-lab9.
2. **Prove the data is readable now.** Before encrypting, create a file with recognizable content ("SENSITIVE-LAB9-DATA") on the endpoint. Power the VM off. Attach its virtual disk to another lab VM as a secondary disk and confirm you can read the file. This is the unencrypted baseline: disk off, data readable by anyone who holds the disk.
3. **Enable full-disk encryption.**
   - Windows: enable BitLocker on the system drive. If the VM lacks a virtual TPM, you can enable BitLocker with a startup password or key instead; the lab is about the data-at-rest protection either way.
   - Linux: the clean install likely offered LUKS encryption; if not, this is a good reason to rebuild LIN-ENDPOINT with LUKS enabled, since full-disk encryption is best set at install time.

Let encryption complete.

1. **Record where the key lives.** Note how the disk gets unlocked at boot: a TPM releasing the key automatically, a password you type, a recovery key. Write down where the recovery key is stored. If your recovery key is saved on the encrypted machine itself, it is protecting nothing; store it somewhere off the machine. This is the key-management discipline.
2. **Prove the at-rest protection.** Power the VM off. Attach its encrypted virtual disk to another lab VM again and try to read "SENSITIVE-LAB9-DATA." You should not be able to; the disk is unreadable without the key. The lost-laptop scenario is now defended.
3. **Prove the limit. This is the heart of the lab.** Boot the encrypted VM and log in, so the disk is unlocked and the user is working, the normal daily state. Now, from that running session (or by accessing the running machine as an attacker would after compromising it), read "SENSITIVE-LAB9-DATA." It is fully readable. Full-disk encryption did nothing here, because the disk is unlocked whenever the machine is on and in use. Record this clearly.

4. **Address removable media.** Configure a policy that controls USB storage, either blocking it or requiring that data written to removable media is itself encrypted. The USB drive is how data walks off the endpoint and how threats walk on.
5. **Restore.**

Lab worksheet

State	Can an attacker read the sensitive file?
Unencrypted, powered off	
Encrypted, powered off	
Encrypted, powered on and unlocked	

Key management: How the disk unlocks at boot: ____ Where the recovery key is stored: ____ Is the recovery key off the machine? ____

Removable media control: Approach taken: ____

Expected outcomes

A clean three-row result: the unencrypted disk is readable at rest, the encrypted disk is unreadable at rest, and the encrypted disk is fully readable while running and unlocked. That third row is the one people forget and the one this lab exists to burn in. You should finish able to say precisely what full-disk encryption protects (the powered-off device) and what it does not (the running one, which is most of the day).

Discussion questions

1. Full-disk encryption protects the device at rest. Most of a device's life is spent powered on and unlocked. What other controls in this course protect the data during that much larger window?
2. A recovery key stored on the machine it recovers is useless, yet it happens constantly. Where should recovery keys actually live in a real organization, and what new risk does that central store create?
3. Data loss prevention watches for sensitive data leaving the endpoint. Based on what you saw, at what moment, on-disk, in-use, or in-transit, does DLP have to operate, and why can encryption not do DLP's job?

Grading rubric

Criterion	Points
Unencrypted at-rest readability demonstrated as a baseline	15
Full-disk encryption enabled successfully	20
At-rest protection proven (encrypted disk unreadable when off)	20
The running-and-unlocked limit demonstrated clearly	25
Key management and recovery-key location addressed correctly	10
Removable-media control configured	10
Total	**100**

Lab 10: Host Firewall with Outbound Control

Companion to Video 10: Network Defense at the Endpoint

Lab objectives

By the end of this lab you will be able to:

- Configure a host firewall on a lab endpoint that controls both inbound and outbound connections.
- Demonstrate how outbound restriction limits a simulated command-and-control connection.
- Explain why outbound control matters as much as inbound, and how endpoint isolation fits the response phase ahead.

Prerequisites

- WIN-ENDPOINT (or LIN-ENDPOINT) from a recent snapshot.
- ATTACK-HOST, which will play the role of the attacker's command-and-control server.
- Snapshot before starting.

Scenario

Most people think of a firewall as something that keeps attackers out. On an endpoint, the more valuable job is often the reverse: controlling what a compromised machine is allowed to connect to. An intrusion that cannot phone home cannot be remotely controlled and cannot exfiltrate data over its preferred channel. This lab builds outbound control and proves it strangles a simulated command-and-control link.

Step-by-step instructions

1. **Snapshot** as pre-firewall-lab10.
2. **Establish the unrestricted baseline.** With the default firewall configuration, simulate command-and-control: on ATTACK-HOST, start a listener on some port. From the endpoint, open an outbound connection to ATTACK-HOST on that port (a simple scripted beacon that connects out every few seconds). Confirm it connects freely. This is a compromised endpoint phoning home with nothing in its way.
3. **Configure inbound control.** Set the host firewall to deny inbound connections by default and allow only what the machine genuinely needs to receive. Confirm normal operation is unaffected.
4. **Configure outbound control. This is the valuable part.** Change the outbound policy from "allow all" to default-deny, then allow only the outbound connections the endpoint legitimately needs (for the lab, allow what a normal user workflow requires and nothing else). This is more restrictive than most endpoints are configured, and that is the point.
5. **Demonstrate the effect on command-and-control.** Re-run the beacon from step 2. The outbound connection to ATTACK-HOST's arbitrary port should now be blocked by the default-deny outbound policy. The compromised endpoint can no longer reach its controller. Record the before and after.
6. **Test the realistic evasion.** Attackers know about outbound filtering, so they beacon over ports and protocols that are usually allowed (such as standard web ports). Point your beacon at a port your allow rules permit and see whether it gets through. This shows the limit of port-based outbound control and previews why later detection has to look at behavior, not just ports.
7. **Practice endpoint isolation.** Configure a firewall rule set that cuts the endpoint off from everything except your MONITOR/management path, a one-command "isolate this machine" state. This is the containment action Lab 16 will use for real. Confirm you can isolate and then restore connectivity.
8. **Restore.**

Lab worksheet

Connection test	Result
Beacon to arbitrary port, default config	
Beacon to arbitrary port, outbound default-deny	
Beacon over an allowed port	

Inbound policy: Default action: ____ Allowed exceptions: ____

Outbound policy: Default action: ____ Allowed exceptions: ____

Isolation: Can you cut the endpoint to management-only and restore it? ____

Expected outcomes

A host firewall that blocks the simulated command-and-control connection once outbound control is in place, and an understanding of exactly where that control stops working, when the attacker uses an allowed port. You should also have a working isolation capability. The combination, outbound restriction plus the ability to fully isolate, is the endpoint's contribution to both prevention and response.

Discussion questions

1. Inbound firewalling is standard; strict outbound filtering is rare. Why do so few endpoints restrict outbound connections, given how much it limits a compromise?
2. Your beacon got through once it used an allowed port. If you cannot block the port, what else about the connection could a defender use to tell malicious traffic from legitimate traffic?
3. Endpoint isolation is powerful and also disruptive, it cuts a user off entirely. Who should be able to trigger it, and what has to be true before you isolate a machine in a real incident?

Grading rubric

Criterion	Points
Unrestricted command-and-control baseline demonstrated	15
Inbound default-deny configured without breaking normal use	15
Outbound default-deny configured with justified exceptions	25
Command-and-control shown blocked after outbound control	20
Allowed-port evasion tested and its lesson recorded	15
Isolation capability built and confirmed reversible	10
Total	**100**

Lab 11: Secure Boot and Hardware-Backed Protections

Companion to Video 11: Physical and Boot-Level Security

Lab objectives

By the end of this lab you will be able to:

- Enable Secure Boot and hardware-backed protections on a lab endpoint.
- Confirm what these controls prevent and, just as importantly, what they do not.
- Explain the chain of trust from power-on and why the earliest code has the most power.

Prerequisites

- WIN-ENDPOINT configured as a UEFI VM with a virtual TPM, if your hypervisor supports one. Most modern hypervisors can provide a virtual TPM and UEFI firmware to the guest.
- Access to the VM's firmware settings.
- Snapshot before starting.

Scenario

An attacker with control of the boot process or physical access can defeat controls that live higher up, because they run before those controls exist. The earliest code to run has the most power and, in most environments, the least protection. This lab enables the protections that anchor the endpoint's integrity in hardware and establishes a chain of trust from the moment of power-on, then honestly marks the boundary of what they cover.

Step-by-step instructions

1. **Snapshot** as pre-boot-lab11.
2. **Observe the starting boot state.** Enter the VM's firmware settings. Note whether Secure Boot is enabled, whether a TPM is present, and whether boot from external media is allowed. This is the physical and boot attack surface as it stands.
3. **Enable Secure Boot.** Turn on Secure Boot in firmware. Boot the machine and confirm it still loads. Secure Boot establishes that only code signed by trusted keys runs from power-on, the chain of trust. Record that the OS still boots (a correctly signed OS should).
4. **Confirm the chain-of-trust effect.** To see what Secure Boot enforces, attempt to boot an unsigned or untrusted boot image from external media (a test ISO). Secure Boot should refuse it. This is the control preventing untrusted early code from running. Record the refusal.
5. **Enable hardware-backed protections.** With the TPM present, confirm that TPM-backed features are available: BitLocker keyed to the TPM (tying disk unlock to the machine's measured boot state), and, on supported Windows, virtualization-based security features. The TPM is the hardware root of trust, something an attacker cannot easily forge.
6. **Reduce the physical surface.** In firmware, set a firmware/BIOS password and disable booting from external media. These address the attacker with brief physical access, the "evil maid" who reboots an unattended machine from their own USB.
7. **Mark the boundary. The honest part.** List what these controls do *not* stop. Secure Boot does not protect a machine that is already booted and unlocked. A firmware password does not stop an attacker who can open the case and reset it. TPM-backed encryption does not help against an attacker who has the running, unlocked machine (the Lab 9 lesson again). Write these down so you do not overcredit the controls you just enabled.
8. **Restore.**

Lab worksheet

Starting state: Secure Boot: ____ TPM present: ____ External boot allowed: ____

Protection enabled	Confirmed working	What it prevents
Secure Boot		
TPM-backed disk encryption		
Firmware password		
External boot disabled		

Chain of trust: Did Secure Boot refuse an untrusted boot image? ____

The boundary, what these do NOT prevent: ____

Expected outcomes

A machine whose boot process is anchored in a hardware root of trust and refuses untrusted early code, with a firmware configuration that resists quick physical attacks. Equally, a clear written boundary of what these controls leave uncovered. The valuable realization is that boot and physical security close a specific and dangerous door, the one below the operating system, while leaving the running-machine doors exactly where the rest of the course addresses them.

Discussion questions

1. The earliest code has the most power and usually the least scrutiny. Why has boot-level security historically been neglected, and what changed to make hardware roots of trust common?
2. Device attestation lets an endpoint prove its own integrity to something else before being trusted. How does that connect boot-level security to the zero-trust and identity ideas from Lab 10 and beyond?
3. A firmware compromise can survive an operating-system reinstall. If reinstalling does not clear it, how would you ever detect or recover from a firmware-level compromise?

Grading rubric

Criterion	Points
Starting boot/firmware state documented	10
Secure Boot enabled and the OS still boots	20
Chain of trust demonstrated by refusing an untrusted image	20
TPM-backed protection confirmed	15
Physical surface reduced (firmware password, external boot disabled)	15
Boundary of these controls stated honestly and correctly	20
Total	**100**

Lab 12: Deploy Endpoint Protection and Evade It

Companion to Video 12: Antivirus and the Evolution to Endpoint Protection

Lab objectives

By the end of this lab you will be able to:

- Deploy endpoint protection (antivirus / endpoint protection platform) to a lab endpoint.
- Confirm it catches commodity malware.
- Demonstrate a technique that evades it, understanding first-hand both the value and the hard limit of prevention.

Prerequisites

- WIN-ENDPOINT from a recent snapshot.
- The EICAR test file and the in-memory technique from Lab 5.
- ATTACK-HOST running.
- Snapshot before starting.

Scenario

Antivirus is much maligned and still stops a large volume of commodity threats every day, which is why dismissing it is a mistake. But it works primarily by recognizing known-bad code, and that creates a fundamental limit against anything new or fileless. This lab has you deploy it, watch it do its real and useful job, and then watch something walk straight past it, which is precisely the realization that motivated detection and response.

Step-by-step instructions

1. **Snapshot** as pre-av-lab12.
2. **Deploy endpoint protection.** On WIN-ENDPOINT, confirm Microsoft Defender Antivirus is enabled and updated, or deploy another endpoint protection product if you prefer. Ensure real-time protection and cloud-delivered protection are on. Cloud lookup lets the product consult global threat reputation data rather than relying only on local signatures.
3. **Confirm it catches commodity malware.** Place the EICAR test file on the machine. Defender should detect and quarantine it immediately. Try a few variations (different folders, inside an archive) and record what is caught. This is signature-based detection doing its job well against known-bad code.
4. **Observe heuristic/behavioral detection.** Run a harmless action that mimics a suspicious behavior pattern (for example, a script that rapidly creates many files, mimicking the shape of ransomware without doing harm) and see whether behavioral detection reacts. This is the move beyond signatures: catching threats by what they do, not only what they are.
5. **Now evade it.** Use the in-memory, living-off-the-land technique from Lab 5, running a payload entirely in memory using a trusted built-in tool, so that no known-bad file ever touches disk for a signature to match. Confirm the action completes without antivirus stopping it. You have just demonstrated the fundamental limit: a control built around recognizing bad files is weak against an attack that never writes one.
6. **Confirm the management and monitoring reality.** Check where the detection from step 3 was recorded. In a real deployment this alert flows to a console someone monitors. Note what would happen to the step-3 alert if nobody was watching the console, this is the "alert nobody monitored" failure the course names.
7. **Record the honest conclusion.** Antivirus caught the commodity threat and missed the fileless one. Both results are correct and expected. Write the one-sentence takeaway: prevention stops volume, not determination.
8. **Restore.**

Lab worksheet

Test	Detected by endpoint protection?
EICAR in a normal folder	
EICAR inside an archive	
Ransomware-shaped behavior (harmless)	
In-memory living-off-the-land technique	

Cloud protection enabled? ____ Where detections are recorded: ____

Evasion technique used: ____ Did it complete without being stopped? ____

One-sentence takeaway: ____

Expected outcomes

Endpoint protection that reliably catches the commodity malware and just as reliably misses the fileless technique. If the EICAR tests were caught and the in-memory technique succeeded, the lab worked exactly as intended. This is not a knock on antivirus; it is the honest shape of prevention, and it is the specific gap that Lab 13's detection and response exists to cover.

Discussion questions

1. Antivirus caught the file-based threats and missed the fileless one. If you removed antivirus entirely, what would get worse? Argue for keeping it despite the gap you just demonstrated.
2. The behavioral detection in step 4 catches threats by what they do. Why can it not simply be turned up high enough to catch everything, what is the cost of making behavioral detection more aggressive?
3. The course names "the antivirus disabled by the malware it should have caught." How could malware disable the very control meant to stop it, and what would prevent that? (Lab 18 addresses tamper protection directly.)

Grading rubric

Criterion	Points
Endpoint protection deployed with real-time and cloud protection on	20
Commodity malware detection confirmed across variations	20
Behavioral detection observed	15
Evasion via a fileless/LOL technique demonstrated	30
Management/monitoring reality and the unmonitored-alert risk noted	10
Honest conclusion articulated	5
Total	**100**

Part III: Detection and Response

Lab 13: Deploy Detection and Response, Then Catch an Attack

Companion to Video 13: Endpoint Detection and Response

Lab objectives

By the end of this lab you will be able to:

- Deploy endpoint detection and response tooling that continuously records endpoint activity.
- Run an attack from Lab 5 against the endpoint and use the recorded telemetry to detect and investigate it.
- Map what you detect to the attacker techniques behind it, and feel the signal-to-noise problem directly.

Prerequisites

- WIN-ENDPOINT from a recent snapshot, with telemetry flowing to MONITOR from Lab 3.
- ATTACK-HOST running.
- Your Lab 5 technique reference.
- Snapshot before starting.

Scenario

Prevention will fail; Lab 12 just proved it. Detection and response is the response to that fact: instead of trying to stop everything, you continuously watch the endpoint and analyze its activity for signs of compromise, so you catch what prevention missed. This lab stands up that continuous recording, then runs the very technique that evaded antivirus and catches it in the telemetry, because what is invisible to a file scanner is often loud in a behavior record.

Step-by-step instructions

1. **Snapshot** as pre-edr-lab13.
2. **Deploy the telemetry/EDR tooling.** For a lab, the open-source combination of Sysmon (detailed endpoint event logging) feeding into a collection point gives you a genuine endpoint detection and response capability without commercial licensing. Install Sysmon on WIN-ENDPOINT with a good configuration (a community-standard config captures process creation with command lines, network connections, file creation, registry changes, and image loads). Confirm its events reach MONITOR.
3. **Understand your telemetry.** Before attacking, look at what the endpoint now records in normal operation: process creations, network connections, file writes, registry modifications. This is the raw material of detection. Note how much of it there is even on an idle machine, your first taste of the signal-to-noise problem.
4. **Run the attack.** From ATTACK-HOST, execute the in-memory living-off-the-land technique from Lab 5, the one antivirus missed in Lab 12. Then run the enumeration sequence and plant a persistence mechanism, so you have a small multi-step intrusion to find.
5. **Detect it in the telemetry.** Go to MONITOR and work backward from the attack you know you ran. Find the process-creation event for the suspicious use of the built-in tool. Find the network connection to ATTACK-HOST. Find the registry or scheduled-task change for persistence. You are using the behavior record to see an attack that left nothing a signature could match.
6. **Map to techniques.** For each thing you found, name the attacker technique it represents. Structuring detection around the techniques attackers actually use, rather than around specific tools, is what makes detection durable; the MITRE ATT&CK framework is the standard catalog of these techniques, and mapping your findings to it is good practice.

7. **Build one detection rule.** Turn one of your findings into a reusable detection: a rule or saved query that would flag this behavior automatically next time (for example, "a built-in scripting tool making an outbound network connection to a non-standard destination"). This is the move from one investigation to ongoing coverage.
8. **Feel the triage problem.** Look at how many events your short attack generated versus how many total events the machine produced in the same window. This ratio, a few malicious events hidden in thousands of benign ones, is the daily reality of detection, and the reason triage and tuning matter as much as the detection itself.
9. **Restore.**

Lab worksheet

Telemetry deployed: Tool(s): ____ Event types captured: ____ Reaching MONITOR? ____

Attack run: Techniques executed: ____

Detection, what you found in the telemetry:

Attacker action	Event that revealed it	Data source (process/network/file/registry)	ATT&CK technique (if known)
In-memory execution			
Command-and-control connection			
Persistence			

Detection rule built: ____

Signal-to-noise: Malicious events: ____ Total events in the window: ____ Ratio: ____

Expected outcomes

You catch, in the behavior record, the attack that walked past antivirus. Every step of your small intrusion should be visible somewhere in the telemetry even though it left little on disk. You should also come away slightly alarmed at the event volume, which is the correct reaction and the reason the next labs cover hunting, tuning, and operations rather than assuming detection alone solves the problem.

Discussion questions

1. The attack that beat antivirus was visible in your telemetry. What specifically made it visible, and what would an attacker have to do to be quiet in the behavior record as well?
2. You mapped findings to attacker techniques rather than to specific tools. Why is technique-based detection more durable than tool-based or signature-based detection?
3. Your malicious events were a tiny fraction of total events. Given that ratio, what goes wrong if an organization simply "turns on all the detections" without tuning, and how does that connect to the alert-fatigue failures the course keeps naming?

Grading rubric

Criterion	Points
Detection/telemetry tooling deployed and events reaching the monitor	25
Normal-operation telemetry examined before attacking	10
Multi-step attack executed in the lab	15
Each attacker action located in the telemetry	25
Findings mapped to techniques and one detection rule built	15

Criterion	Points
Signal-to-noise ratio measured and reflected on	10
Total	**100**

Lab 14: Hunt for Persistence and Turn a Finding into a Detection

Companion to Video 14: Threat Hunting on the Endpoint

Lab objectives

By the end of this lab you will be able to:

- Develop a hypothesis-driven hunt for persistence across your lab endpoints.
- Execute the hunt using your endpoint telemetry, finding attacker persistence that no alert fired on.
- Turn a hunt finding into an automated detection, closing the loop so the same thing is caught by alert next time.

Prerequisites

- Lab 13 completed; telemetry tooling in place on WIN-ENDPOINT, and ideally on LIN-ENDPOINT too, so you can hunt across more than one machine.
- A partner or your own earlier self to plant persistence you then hunt for (see step 1).
- Snapshot before starting.

Scenario

Threat hunting goes beyond waiting for alerts: you actively search for compromise that detection missed, starting from a specific idea of what an attacker would do. Hunting is where you find the intrusions that were quiet enough to avoid every alert. This lab runs a real hunt for persistence, the mechanisms attackers use to survive a reboot, and then does the thing that makes hunting pay off over time: it turns the finding into a standing detection.

Step-by-step instructions

1. **Plant the quarry (or have someone else do it).** For a real hunt, you should not know exactly what you are looking for. Ideally, have someone else plant one or two persistence mechanisms on WIN-ENDPOINT and LIN-ENDPOINT from the Lab 5 set (a Run key, a scheduled task, a cron job, a service), without telling you which. If working alone, plant several, wait a day so the details fade, then hunt. Snapshot after planting.
2. **State the hypothesis.** Write the hunt hypothesis before you touch the telemetry. Example: "An attacker who gained access to these endpoints would establish persistence via an autostart mechanism (registry Run keys, scheduled tasks, services, or cron) so they survive a reboot. If so, I will find an autostart entry that launches an unusual or user-writable-path binary or script." A hunt without a hypothesis is just aimless clicking, one of the hunting failures the course names.
3. **Collect the persistence locations.** For each endpoint, enumerate the autostart locations:
   - Windows: Run and RunOnce keys, scheduled tasks, services, startup folders. Tools like Autoruns make this systematic.
   - Linux: cron (`crontab -l`, `/etc/cron*`), systemd units and timers, `.bashrc`/profile scripts, rc scripts.
1. **Hunt.** Go through the enumerated entries and separate the normal from the suspicious. You are looking for entries that launch from user-writable locations, have odd names, were created recently, or point at scripts rather than installed software. Cross-reference against your telemetry from Lab 13 to see when each entry was created and by what process. This is the hunt: hypothesis plus telemetry plus judgment.
2. **Confirm findings.** For each suspicious entry, determine whether it is actually the planted persistence or a legitimate entry you did not recognize. Record how you told the difference. Hunting produces both findings and false leads, and distinguishing them is the skill.
3. **Hunt for living-off-the-land persistence too.** Some persistence abuses legitimate tools rather than adding an obviously foreign file. Check for scheduled tasks or services that run built-in interpreters with suspicious arguments. This is the harder, quieter class.
4. **Turn the finding into a detection.** Take one confirmed finding and write an automated detection for it: a rule or scheduled query that flags "a new autostart entry pointing to a user-writable path" or "a scheduled task

launching a scripting interpreter with network arguments." Now the next instance alerts automatically. This closing of the loop, hunt becomes detection, is what separates mature hunting from one-off searches.

5. **Restore.**

Lab worksheet

Hypothesis (written before hunting): ____

Autostart locations enumerated:

Endpoint	Locations checked	Entries found (count)	Suspicious entries
WIN-ENDPOINT			
LIN-ENDPOINT			

Findings:

Suspicious entry	Legitimate or planted persistence?	How I told the difference

Detection created from a finding: ____

Expected outcomes

You find the planted persistence by reasoning from a hypothesis, not by waiting for an alert, and you create a detection so the same technique would alert next time. You should also generate at least one false lead, a legitimate autostart entry that looked suspicious, because learning to clear those quickly is as important as finding the real one. The finished detection rule is the lasting artifact.

Discussion questions

1. You wrote a hypothesis before hunting. How did having it focus your search, and what would have happened if you had just started "looking around" the telemetry instead?
2. Distinguishing planted persistence from legitimate-but-unfamiliar entries took judgment. What reference would make that faster, and how would you build it for your own environment?
3. A hunt finding that never becomes a detection is a hunt you will have to repeat forever. Beyond the one rule you wrote, what other findings from today should become standing detections, and what is the cost of turning every finding into an alert?

Grading rubric

Criterion	Points
Clear, specific hunt hypothesis written before hunting	20
Autostart/persistence locations systematically enumerated on both endpoints	20
Planted persistence found through the hunt	25
Suspicious entries confirmed versus cleared, with reasoning	15
Living-off-the-land persistence considered	5
A finding converted into an automated detection	15
Total	**100**

Lab 15: Investigate a Compromised Endpoint

Companion to Video 15: Endpoint Forensics and Investigation

Lab objectives

By the end of this lab you will be able to:

- Investigate a compromised lab endpoint in depth, capturing volatile evidence before it disappears.
- Find the attacker's persistence and reconstruct a timeline of what happened and when.
- Apply the order-of-volatility discipline and scope the compromise from one endpoint outward.

Prerequisites

- WIN-ENDPOINT, which you will deliberately compromise and then investigate.
- The Lab 5 techniques and ATTACK-HOST.
- Basic forensic tooling: a memory-capture tool and the ability to examine captured memory, plus the ability to read endpoint artifacts. Open-source tools suffice for the lab.
- Snapshot before starting.

Scenario

When an endpoint is compromised, understanding exactly what happened drives everything that follows: how far it spread, what was taken, how to be sure you got it all. This lab runs a full investigation of a machine you compromise on purpose. The governing discipline is order of volatility: the most fragile evidence, the contents of memory, disappears the instant the machine powers off, so you capture it first, before you do anything else.

Step-by-step instructions

1. **Snapshot and compromise.** Snapshot WIN-ENDPOINT as `pre-forensics-lab15`. Then run a multi-step intrusion against it: an in-memory payload, enumeration, and persistence, leaving the machine in a "running, compromised" state. Do not reboot it. Treat it from here on as an incident.
2. **Capture volatile evidence first.** Before anything else, capture the machine's memory while it is still running. Also record the volatile state: running processes, open network connections, logged-on users, loaded modules. This is the evidence that is gone the moment the machine is shut down, which is why it comes first. Record exactly what you captured and when.
3. **Hunt for the fileless threat in memory.** Analyze the captured memory for the in-memory payload, the one that left little or nothing on disk. Memory analysis is how you find injected and fileless threats that disk forensics would miss entirely. Record what you found in memory that was not visible on disk.
4. **Collect the on-disk artifacts.** Now gather the execution evidence and attacker traces: process execution artifacts, recently created files, the persistence mechanisms, relevant logs. These corroborate and extend what memory told you.
5. **Build the timeline.** Combine everything, memory findings, file timestamps, log events, persistence creation times, into a single timeline: what the attacker did, in order, with times. The timeline is the deliverable that makes the incident understandable. Reconstruct the intrusion from first action to last.
6. **Scope from this one endpoint.** Ask what this compromised machine reveals about others. Did the attacker's activity reference credentials, network connections, or targets that implicate WIN-SERVER or LIN-ENDPOINT? In a real incident, one compromised endpoint is how you find the rest; underestimating scope leaves infected machines in place. Record what you would check on other machines based on this one's evidence.
7. **Note the response-versus-forensics tension.** You have been preserving evidence. A real incident also demands fast recovery, and the two pull against each other, wiping the machine quickly destroys the evidence you just collected. Write down how you balanced them and what you would do differently under time pressure. (Lab 16 is the response side of this coin.)

8. **Restore.**

Lab worksheet

Volatile capture (in order of volatility):

Evidence type	Captured?	Time captured
Memory image		
Running processes		
Network connections		
Logged-on users / sessions		

Memory findings not visible on disk: ____

On-disk artifacts collected: ____

Timeline (attacker actions in order):

Time	Action	Evidence source

Scoping: What this endpoint suggests to check on other machines: ____

Response-vs-forensics: How you balanced preservation against recovery: ____

Expected outcomes

A full investigation that recovers the fileless payload from memory (where disk forensics alone would have missed it), identifies the persistence, and produces a coherent timeline of the intrusion. The key realization is order of volatility made concrete: had you shut the machine down first, as an untrained responder might, the memory evidence, and with it the clearest trace of the fileless attack, would have been gone.

Discussion questions

1. The fileless payload showed up in memory but barely on disk. If an untrained person had powered the machine off to "stop the attack," what evidence would have been lost, and why is "pull the plug" such a damaging instinct here?
2. Your timeline turned scattered artifacts into a story. Which single evidence source contributed most to the timeline, and what would the investigation have looked like without it?
3. Scoping from one endpoint is how incidents are sized correctly. What is the risk of declaring an incident over based on cleaning just the one machine you found first?

Grading rubric

Criterion	Points
Volatile evidence captured first, in correct order	25
Memory analyzed and the fileless threat recovered	20
On-disk artifacts and persistence collected	15
Coherent timeline reconstructed	20
Scoping to other endpoints reasoned through	10
Response-versus-forensics tension addressed	10
Total	**100**

Lab 16: Run a Full Endpoint Incident

Companion to Video 16: Responding to Endpoint Incidents

Lab objectives

By the end of this lab you will be able to:

- Run a compromised endpoint through the full incident lifecycle: detection, containment, eradication, and recovery.
- Contain a machine without destroying the evidence the investigation needs.
- Decide, with reasons, whether to clean a compromised endpoint or wipe and rebuild it.

Prerequisites

- WIN-ENDPOINT, the isolation capability you built in Lab 10, and the investigation skills from Lab 15.
- A clean baseline snapshot available for the rebuild option.
- Snapshot before starting.

Scenario

Investigation tells you what happened; response is what you do about it. This lab takes a compromised endpoint through the whole incident lifecycle under the two tensions that define endpoint response: containing the machine without destroying evidence, and deciding whether cleaning is enough or whether only a rebuild can be trusted. You will make that rebuild-versus-clean call for real.

Step-by-step instructions

1. **Snapshot and compromise.** Snapshot as pre-response-lab16. Compromise WIN-ENDPOINT with a multi-step intrusion including persistence, as in Lab 15.
2. **Detect and declare.** Using your telemetry, detect the compromise and formally "declare the incident", mark the start time and note what triggered the detection. The lifecycle begins here.
3. **Contain without destroying evidence.** Isolate the endpoint using the firewall isolation from Lab 10, cutting the attacker's access while keeping the machine running so you do not lose volatile evidence. This is the balance: stop the spread, preserve the scene. Capture volatile evidence now if you have not already (order of volatility still applies). Record the containment action and time.
4. **Investigate enough to eradicate.** You cannot remove what you have not found. Using Lab 15's methods, identify every piece of the attacker's presence: the payload, the persistence, any accounts or changes they made. Eradication that misses one foothold invites reinfection.
5. **Make the clean-versus-rebuild decision.** Decide: can you confidently clean this machine, removing every attacker artifact, or is the compromise deep enough that a wipe and rebuild is the only trustworthy option? Write your reasoning. The course's position is that a thoroughly compromised endpoint is often safer wiped than cleaned, because you can never fully prove a clean removed everything. State what, in your evidence, drives your choice.
6. **Execute the choice.**
   - If cleaning: remove every identified artifact and persistence mechanism, then verify none remain.
   - If rebuilding: wipe and restore from the clean baseline (this is where the virtualization discipline pays off directly).

Record which you did.

1. **Recover.** Return the endpoint and its user to safe, normal operation: restore needed data, re-apply your hardening and controls, confirm the user can work. Note that a rebuilt machine needs all your earlier hardening re-applied, which is an argument for the fleet automation in Part IV.

2. **Verify no reinfection.** Run the machine and watch the telemetry. Confirm the persistence does not re-trigger and the attacker does not return. A response that leaves a foothold shows up here.
3. **Confront the scale problem.** Write one paragraph: everything you just did was for one endpoint. How would this change if it were a hundred endpoints at once? This previews Part IV directly.

Lab worksheet

Lifecycle phase	Action taken	Time	Notes
Detection / declaration			
Containment			
Investigation / eradication			
Recovery			

Volatile evidence preserved before/during containment? ____

Clean vs rebuild decision: Choice: ____ Reasoning: ____

Verification: Persistence gone? Attacker returned? ____

Scale reflection: ____

Expected outcomes

A compromised endpoint taken cleanly through the full lifecycle, with evidence preserved through containment and a defensible clean-versus-rebuild decision. If you chose to rebuild, the restore from baseline should be fast and total, demonstrating why wiping is often the trustworthy choice. You should finish aware that doing this well for one machine is demanding, and that doing it for many at once is a different problem the course has not solved yet.

Discussion questions

1. You contained the machine while keeping it running to preserve evidence. When might that be the wrong call, when would you prioritize immediate shutdown over evidence?
2. Argue the opposite of your clean-versus-rebuild choice. What would have to be different about the compromise to change your decision?
3. Recovery included re-applying your hardening to the rebuilt machine, by hand. In an incident spanning many machines, what breaks about doing recovery by hand, and what from Part IV would fix it?

Grading rubric

Criterion	Points
Incident detected and formally declared	10
Containment achieved while preserving evidence	25
Attacker presence fully identified for eradication	20
Clean-versus-rebuild decision made with sound reasoning	20
Recovery completed and controls re-applied	15
Reinfection verified against, scale reflection written	10
Total	**100**

Lab 17: Plant Deception and Catch an Attacker With It

Companion to Video 17: Deception and Advanced Detection

Lab objectives

By the end of this lab you will be able to:

- Plant honey credentials and honey files on a lab endpoint.
- Trigger them with a simulated attacker and confirm the high-fidelity alert they produce.
- Explain why deception generates almost no false positives, and where a skilled attacker might detect and avoid it.

Prerequisites

- WIN-ENDPOINT with telemetry flowing to MONITOR.
- ATTACK-HOST playing the attacker.
- Snapshot before starting.

Scenario

Most detection fights a signal-to-noise battle, telling malicious activity apart from mountains of legitimate activity. Deception sidesteps that battle. You plant bait that only an attacker would ever touch, so when it is touched, the alert is almost certainly real. This lab plants that bait, springs it, and confirms the clean, high-confidence signal, then considers how a careful adversary tries to spot the trap.

Step-by-step instructions

1. **Snapshot** as pre-deception-lab17.
2. **Plant honey credentials.** Create credentials that look valuable but are fake and unused, for example, a convincingly named account in a location an attacker would look (saved credentials, a "passwords.txt" on the desktop, a fake admin account in the directory). The defining property: no legitimate user or process ever uses these, so any use is an intruder.
3. **Plant honey files.** Create files that look like attractive targets, a document named like sensitive financial or credential data, placed where an attacker rummaging the machine would find it. Configure auditing so that any access to these files is logged.
4. **Wire the alert.** Configure your telemetry so that any use of the honey credentials or any access to the honey files produces an alert on MONITOR. Because legitimate activity never touches these, the alert can be high-priority by default.
5. **Spring the trap.** From ATTACK-HOST, simulate an attacker who has landed on WIN-ENDPOINT and is looking for loot: enumerate saved credentials and find the honey credential, open the honey file. Then attempt to use the honey credential to authenticate somewhere.
6. **Confirm the high-fidelity alert.** On MONITOR, confirm the trap fired. Compare the clarity of this alert to the behavioral detections from Lab 13: this one needs no triage to believe, because nothing benign could have caused it. Record the difference in confidence.
7. **Think like the attacker.** A sophisticated adversary tries to fingerprint and avoid deception, credentials that are never used, files that are never opened by the real user, accounts with no login history can all look like bait to a careful attacker. Note how your honey objects might give themselves away, and what would make them more convincing (and what the course warns about: a honey token that reveals too much about the environment).
8. **Layer it.** Write one sentence on how you would combine this high-fidelity, low-coverage deception with the broad, noisier behavioral detection from Lab 13 so that together they cover both "quiet attacker who trips a trap" and "loud attacker who does not."
9. **Restore.**

Lab worksheet

Honey objects planted:

Type	What it looks like	Where placed	Auditing/alerting wired?
Honey credential			
Honey file			

Trap sprung: Attacker actions taken: ____ Alert fired on MONITOR? ____

Fidelity comparison: Confidence in this alert vs a Lab 13 behavioral alert: ____

Attacker's view: How your honey objects might be fingerprinted: ____

Layering plan: ____

Expected outcomes

A trap that fires a clean, immediately believable alert the moment the simulated attacker touches it. The contrast with Lab 13's behavioral detection should be striking: no triage, no "is this real?", because nothing legitimate could have triggered it. You should also come away with a healthy sense that deception is powerful precisely where it is convincing, and useless the moment an attacker recognizes it as a trap.

Discussion questions

1. Deception produces almost no false positives. Given that, why not rely on it as the primary detection method, what does its near-zero false-positive rate cost it in coverage?
2. A skilled attacker tries to avoid bait. What design choices make a honey credential or file hard to distinguish from the real thing, and what is the risk of making them too realistic?
3. The course warns of "the honey token that revealed too much about the environment." How could a trap meant to catch an attacker instead hand them useful information?

Grading rubric

Criterion	Points
Honey credentials and honey files planted convincingly	25
Alerting correctly wired to any interaction with them	20
Trap sprung by the simulated attacker	20
High-fidelity alert confirmed and contrasted with behavioral detection	20
Attacker's-eye analysis of how deception could be detected	10
Layering plan articulated	5
Total	**100**

Lab 18: Evade Your Own Controls, Then Add the Layer That Catches It

Companion to Video 18: Evasion and How Endpoint Controls Get Bypassed

Lab objectives

By the end of this lab you will be able to:

- Attempt to evade the endpoint controls you deployed in earlier labs.
- Add the layering and tamper protection that would catch or prevent the evasion.
- Explain, from direct experience, why no single control is enough and how understanding evasion proves the case for defense in depth.

Prerequisites

- WIN-ENDPOINT carrying the controls you built across earlier labs: hardening (6), reduced privilege (7), application control (8), antivirus (12), and detection/telemetry (13).
- ATTACK-HOST.
- Snapshot before starting.

Scenario

You cannot build layered defense without knowing how each layer is bypassed. This lab is the adversarial capstone of Part III: you deliberately try to defeat your own stack, control by control, and then for each successful evasion you add the layer that would have caught it. The payoff is the course's central principle made unavoidable, understanding evasion is what proves no single control holds.

Step-by-step instructions

1. **Snapshot** as pre-evasion-lab18. Inventory the controls currently on the machine so you know what you are trying to beat.
2. **Evade detection with living-off-the-land.** Attempt an action using only built-in, trusted tools, so controls looking for foreign code have nothing to match. You did a version of this in Labs 8 and 12; here, note which of your controls see it and which are blind. Record what got past what.
3. **Attempt to tamper with the security tools.** Try, as an attacker with the privileges you can obtain, to disable or blind your own controls: stop the telemetry agent, disable the antivirus, clear the logs. Note which succeed. A control an attacker can switch off is a control that is not really there, and this is the "security tool the malware disabled" failure.
4. **Attempt an evasion against application control.** Revisit your Lab 8 allowlist and try to run code through an approved tool again. Confirm whether your current configuration still allows it.
5. **Now defend the defenses.** For each successful evasion, add the layer that addresses it:
   - For tampering: enable tamper protection on the antivirus/EDR so it cannot be casually disabled, and protect or forward logs off the machine in real time so clearing local logs does not erase the evidence (your MONITOR already helps here, local log clearing does not reach logs already shipped).
   - For living-off-the-land: add the behavioral detection and the honey-object tripwires from Labs 13 and 17, so the quiet technique that evades prevention still produces a signal.
   - For the application-control bypass: tighten the rules around the abused tool, or add detection for its suspicious use.
1. **Re-test.** Run each evasion again against the newly layered configuration. For each, record whether it is now prevented, or still succeeds but is now detected. "Still succeeds but now detected" is a win, catching what you cannot prevent is the whole premise of Part III.

2. **State the payoff.** Write the conclusion in your own words: for each evasion, which single control failed, and which other layer caught or stopped it. This table is the argument for defense in depth, built from your own machine.
3. **Restore.**

Lab worksheet

Evasion attempt	Got past (before layering)	Layer added	Result after layering (prevented / detected / still succeeds)
Living-off-the-land execution			
Disable telemetry agent			
Disable antivirus			
Clear local logs			
Application-control bypass			

Tamper protection enabled? ____ Logs forwarded off-machine in real time? ____

The defense-in-depth payoff (which layer caught what single control missed): ____

Expected outcomes

A machine where several evasions that succeeded against a single control are now either prevented or at least detected by another layer, and tamper protection that stops the attacker from simply switching your defenses off. The finished table is the point of the whole part: you can show, from your own evasions, that every single control had a bypass and that layering is what turned each bypass into a prevented or detected event rather than a silent success.

Discussion questions

1. Which of your controls was easiest to tamper with, and what does that say about trusting any control that can be disabled from the endpoint it runs on?
2. Forwarding logs off-machine defeats local log clearing. What is the general principle there, and where else in your stack does moving something off the endpoint make it harder to tamper with?
3. Some evasions you could only detect, not prevent. Is a detected-but-successful attack an acceptable outcome? Under what conditions is "we can't stop it but we'll always see it" good enough, and when is it not?

Grading rubric

Criterion	Points
Multiple evasions attempted against the existing control stack	20
Tampering with security tools attempted and results recorded	15
For each successful evasion, an appropriate layer added	25
Tamper protection and off-machine log forwarding implemented	15
Re-test shows evasions now prevented or detected	15
Defense-in-depth payoff articulated from the student's own results	10
Total	**100**

Part IV: Managing Endpoints at Scale

Lab 19: Manage a Fleet from a Central Platform and Catch Drift

Companion to Video 19: Endpoint Management at Scale

Lab objectives

By the end of this lab you will be able to:

- Manage your lab endpoints from a central platform rather than one machine at a time.
- Enforce a hardened baseline across all of them consistently.
- Detect an endpoint that has drifted from the baseline, the configuration divergence that silently opens holes.

Prerequisites

- WIN-ENDPOINT, WIN-SERVER, and LIN-ENDPOINT all running.
- A central management approach. For Windows, Active Directory with Group Policy on WIN-SERVER is the classic small-scale fleet manager and is ideal for this lab; Joshua's homelab already runs AD, so this maps directly to a familiar setup. For cross-platform, a configuration management tool (such as Ansible) managing all three endpoints works and generalizes better.
- Snapshot all machines before starting.

Scenario

Defending one endpoint is nothing like defending a fleet. Everything you did by hand in Parts II and III has to become consistent, enforced, and visible across every machine, or it does not hold. The failure that defines this lab is drift: endpoints quietly diverging from their secure baseline until one of them has a hole nobody meant to open. This lab makes the baseline central and enforced, then catches a drifted machine.

Step-by-step instructions

1. **Snapshot** all three endpoints.
2. **Build the central management.** Stand up your chosen platform:
   - Group Policy route: join WIN-ENDPOINT (and WIN-SERVER as needed) to the Active Directory domain on WIN-SERVER. Now one place governs many machines.
   - Configuration management route: set up Ansible on MONITOR or WIN-SERVER with all three endpoints as managed hosts.
1. **Inventory the fleet.** Confirm the platform knows about every endpoint. You cannot secure the endpoints you do not know you have, so the inventory is the foundation. Record every machine the platform can see, and note any you had to add manually.
2. **Enforce the baseline centrally.** Take a slice of the hardening from Lab 6 and push it to all endpoints from the central platform rather than configuring each by hand, for example, a password policy, an audit policy, and a key security setting. Confirm it applied to every machine. This is the point of fleet management: configure once, enforce everywhere.
3. **Deploy the agent centrally.** Push your telemetry/EDR agent (Sysmon or equivalent) to all endpoints from the platform, rather than installing it machine by machine. Confirm every endpoint is now reporting.
4. **Introduce drift.** On one endpoint only, change a setting away from the baseline, as if a user or a hurried admin had, disable the audit policy, or re-enable a service you had removed, or turn off the agent. This is a single drifted machine in your small fleet.

5. **Detect the drift.** From the central platform, detect which endpoint no longer matches the baseline. With Group Policy, run a resultant-set-of-policy or compliance check; with Ansible, run the playbook in check mode and look for the host reporting changes needed. The platform should flag the one machine that diverged. Record how you found it and how long it took.
6. **Correct the drift.** Re-enforce the baseline from the platform and confirm the drifted endpoint is brought back into line automatically. This closes the loop: central detection plus central correction.
7. **Check the agent-reporting problem.** Confirm you can tell, centrally, that every agent is still reporting. An agent that silently stopped is an endpoint you have gone blind on, and it should itself be a detectable condition.

Lab worksheet

Fleet inventory:

Endpoint	Visible to platform?	Agent reporting?
WIN-ENDPOINT		
WIN-SERVER		
LIN-ENDPOINT		

Baseline enforced centrally: Settings pushed: ____ Applied to all endpoints? ____

Drift introduced on: ____ **Change made:** ____

Drift detection: How detected: ____ Time to detect: ____

Correction: Baseline re-enforced and machine back in line? ____

Expected outcomes

A small fleet governed from one place: baseline pushed to all, agent deployed to all, and a single drifted machine that the platform flags and then corrects. The realization to carry forward is that at scale, security is not a set of settings but a continuous process of enforcement and drift-correction, and that the endpoint you are not managing is the one that will hurt you.

Discussion questions

1. You introduced drift on one machine out of three and caught it. How does the drift problem change when it is one machine out of three thousand, and what makes manual checking impossible at that scale?
2. An unmanaged endpoint is invisible to all of this. How would an endpoint come to be unmanaged in a real organization, and how would you discover it if your platform does not know it exists?
3. The agent that silently stops reporting looks, to the console, a lot like a quiet, healthy machine. How would you design monitoring so that silence is treated as a problem rather than as good news?

Grading rubric

Criterion	Points
Central management platform stood up over all endpoints	25
Fleet inventory confirmed complete	10
Baseline enforced centrally across all machines	20
Agent deployed centrally and reporting confirmed	10
Drift introduced and detected from the platform	20
Drift corrected centrally; agent-reporting check addressed	15
Total	**100**

Lab 20: Scan, Prioritize, and Patch the Fleet

Companion to Video 20: Patch and Vulnerability Management for Endpoints

Lab objectives

By the end of this lab you will be able to:

- Scan your lab endpoints for vulnerabilities across the fleet.
- Prioritize the findings by exploitation risk rather than by raw severity alone.
- Deploy patches across the fleet with verification, and reason about the endpoint that cannot be patched.

Prerequisites

- The managed fleet from Lab 19.
- A vulnerability scanner that runs in a lab (open-source options such as OpenVAS/Greenbone work; vendor community editions also exist).
- At least one endpoint deliberately left unpatched or running an outdated application, so there is something to find.
- Snapshot before starting.

Scenario

The unpatched endpoint undoes much of the hardening around it; a hole in old software is a hole no amount of configuration closes. Patching is one of the highest-impact and most neglected controls. This lab runs the patch-and-vulnerability lifecycle across your fleet, and it insists on the discipline that separates good patching from busywork: patching what is actually being exploited first, not just what has the scariest raw score.

Step-by-step instructions

1. **Snapshot** the fleet. Confirm at least one endpoint has a known outdated component to discover.
2. **Scan the fleet.** Run the vulnerability scanner against all three endpoints. Let it enumerate missing patches and vulnerable software. This is finding the weaknesses before an attacker does. Record the raw findings per endpoint.
3. **Read the results critically.** Scanners produce long lists with severity scores. Note how many findings there are and resist the urge to treat the list top-to-bottom by severity score alone.
4. **Prioritize by exploitation risk.** Re-rank the findings using exploitation intelligence: which vulnerabilities are actually being exploited in the wild, not just which have high theoretical severity. Public resources like the known-exploited-vulnerabilities catalogs capture this. A medium-severity bug under active exploitation outranks a high-severity bug nobody is using. Produce a patch order based on real-world risk.
5. **Account for third-party and application patching.** Note which findings are operating-system patches and which are third-party applications. The operating system is only part of the vulnerable surface, and third-party apps are the ones most often forgotten. Make sure your priority list includes them.
6. **Test before deploying.** Pick the top-priority patch and note the testing-versus-speed tension: deploy too fast and a bad patch can break the fleet; deploy too slow and the vulnerability stays open. For the lab, apply the patch to one endpoint first, confirm nothing broke, then proceed.
7. **Deploy across the fleet with verification.** Using your central platform from Lab 19, deploy the prioritized patches to all applicable endpoints. Then verify, do not assume, that each endpoint actually received and applied each patch. "Deployed" and "applied and confirmed" are different things, and the gap between them is where vulnerabilities quietly survive.
8. **Handle the unpatchable endpoint.** Designate one endpoint (imagine it runs software that cannot be updated) as unpatchable. Since you cannot patch it, design compensating controls instead: isolate it on the network, restrict what it can reach and what can reach it, watch it more closely. Record the compensating controls. (Lab 24 develops this for devices that cannot run an agent at all.)

9. **Re-scan to confirm.** Run the scanner again and confirm the patched vulnerabilities are gone and the compensating-controls endpoint is at least contained.

Lab worksheet

Raw scan findings:

Endpoint	Total findings	Highest raw severity
WIN-ENDPOINT		
WIN-SERVER		
LIN-ENDPOINT		

Prioritized patch order (by exploitation risk, not raw severity):

Rank	Finding	Raw severity	Actively exploited?	OS or third-party
1				

Deployment and verification:

Patch	Deployed to	Applied + verified?

Unpatchable endpoint: Compensating controls chosen: ____

Re-scan: Patched vulnerabilities gone? ____

Expected outcomes

A fleet scanned, a prioritized patch list that reflects what is actually exploited rather than only what scores highest, patches deployed and verified across machines, and a documented compensating-control strategy for the endpoint that cannot be patched. The durable lesson is the prioritization: there will always be more vulnerabilities than time, and patching by real-world exploitation risk is how you spend that limited time where it actually reduces the chance of compromise.

Discussion questions

1. You re-ranked findings by exploitation risk instead of raw severity. Give an example from your scan where the two orders differed, and explain why following raw severity alone would have wasted effort.
2. "Deployed" and "verified applied" came apart as distinct steps. What are the ways a patch can be reported as deployed but not actually be protecting the machine?
3. The unpatchable endpoint got compensating controls instead of a patch. Is a well-isolated unpatched machine more or less risky than a patched machine on a flat network? Defend your answer.

Grading rubric

Criterion	Points
Fleet scanned and raw findings recorded per endpoint	20
Findings prioritized by exploitation risk with reasoning	25
Third-party versus OS patching distinguished	10
Patches deployed across the fleet and verified as applied	25
Compensating controls designed for the unpatchable endpoint	15

Criterion	Points
Re-scan confirms remediation	5
Total	**100**

Lab 21: Write and Enforce an Endpoint Policy with Conditional Access

Companion to Video 21: Endpoint Security Policy and Compliance

Lab objectives

By the end of this lab you will be able to:

- Write an endpoint security policy that states how endpoints must be configured and protected.
- Enforce the policy technically across your lab fleet.
- Configure conditional access that blocks a non-compliant endpoint from a resource it wants.

Prerequisites

- The managed fleet from Lab 19.
- A resource to protect (a lab file share or web service on WIN-SERVER works) and a way to gate access to it on device compliance.
- Snapshot before starting.

Scenario

Consistent, enforceable rules are what make fleet-wide security possible; without them, every machine is a negotiation. This lab moves from configuration to governance: you write the documented standard, enforce it with technical controls, and then wire up the control that gives policy teeth, conditional access, which refuses resources to an endpoint that does not meet the policy. The failure to design out is the policy that is true only on the day of the audit.

Step-by-step instructions

1. **Snapshot** the fleet.
2. **Write the endpoint security policy.** Produce a short, real policy document stating how endpoints in your environment must be configured and protected. Draw on the controls you have built: disk encryption required, standard-user accounts, host firewall on, endpoint protection and telemetry agent running, current patch level, screen lock. Keep it to requirements that can actually be checked. A policy you cannot enforce is the first failure the course names.
3. **Define the compliance state.** For each policy requirement, define what "compliant" looks like as something a machine can prove: encryption on, agent running and reporting, patch level within N days, firewall enabled. This is the device compliance state, how an endpoint demonstrates it meets the policy.
4. **Enforce technically.** Using your central platform, enforce as many policy requirements as the platform can enforce directly (settings pushed and locked). For the rest, create a compliance check that evaluates each endpoint against the policy and reports pass or fail per requirement.
5. **Build conditional access.** Set up access to your protected resource so that it checks the endpoint's compliance state first. A compliant endpoint gets in; a non-compliant one is denied. In the lab this can be as simple as gating the resource behind a check that the endpoint is reporting compliant, the concept is what matters: resource access conditioned on device health.
6. **Test with a compliant endpoint.** From a fully compliant endpoint, access the resource. It should succeed. Record it.
7. **Break compliance and test again.** On one endpoint, violate the policy, turn off the firewall, or stop the agent, so it falls out of compliance. Try to access the protected resource from it. It should now be denied. This is conditional access doing its job: the non-compliant endpoint is not trusted, no matter who is using it. Record the denial.
8. **Address the audit-evidence problem.** Produce the evidence an auditor would want: a report showing each endpoint's compliance state against each policy requirement, as of now. Note how this continuous evidence differs from compliance that is arranged only for audit day.

Lab worksheet

Policy requirements (checkable):

Requirement	How compliance is proven	Enforced how

Conditional access test:

Endpoint state	Access to protected resource
Fully compliant	
Non-compliant (policy broken)	

What was broken to cause non-compliance: ____

Audit evidence produced: Per-endpoint compliance report attached? ____

Expected outcomes

A written policy whose requirements are all checkable, technical enforcement across the fleet, and a working conditional-access gate that admits the compliant endpoint and refuses the non-compliant one. The moment that makes the lab land is watching the same resource be available from a healthy machine and denied from the one whose firewall you just turned off, with the user identity unchanged. Policy stops being a document and becomes a control.

Discussion questions

1. You deliberately wrote only requirements that can be checked. What happens to a policy requirement that sounds good but cannot be technically verified, and how do such requirements let the "true only on audit day" failure creep in?
2. Conditional access denied a machine based on its health, not its user. How does tying resource access to device compliance connect endpoint security to the identity and zero-trust ideas the course keeps referencing?
3. The compliance report is continuous evidence. Why is an organization that can show its compliance state at any moment in a fundamentally stronger position than one that assembles evidence just before an audit?

Grading rubric

Criterion	Points
Endpoint security policy written with only checkable requirements	20
Compliance state defined per requirement	15
Policy enforced technically across the fleet	20
Conditional access configured against the protected resource	20
Compliant admitted and non-compliant denied, both demonstrated	20
Audit-evidence report produced	5
Total	**100**

Lab 22: Enroll and Protect a Mobile Endpoint

Companion to Video 22: Mobile Device Security and Management

Lab objectives

By the end of this lab you will be able to:

- Enroll a mobile device or emulator in a management platform.
- Apply work-data protection that separates and secures work data on a device you do not fully control.
- Demonstrate a remote wipe of only the work data, respecting the personal side of a bring-your-own-device situation.

Prerequisites

- A mobile device you own, or a mobile emulator (Android emulator is easy to run; iOS simulators are more limited for management features).
- Access to a mobile device management platform. Several vendors offer free tiers suitable for a single-device lab; the concepts transfer across platforms.
- Snapshot/backup of the test device or a disposable emulator instance.

Scenario

Phones and tablets are endpoints with a different threat model: they are easily lost, often personally owned, and you rarely control the whole device. Managing them means protecting work data without seizing the user's personal device. This lab enrolls a mobile endpoint, containerizes the work data, and then performs the control that defines the privacy balance, wiping only the work data and leaving the personal side untouched.

Step-by-step instructions

1. **Prepare the device.** Use a disposable emulator or a personal test device you have backed up. Treat it as a bring-your-own-device case: it has both "personal" content and will receive "work" content.
2. **Enroll in management.** Enroll the device in your chosen mobile device management platform. Observe what the platform can and cannot see and control, and note where the line sits. Enrollment is the device agreeing to be managed in exchange for access to work resources.
3. **Apply baseline mobile controls.** Require device encryption (modern mobile OSes encrypt by default, confirm it), require a passcode/biometric, and set the controls appropriate to a device that is easily lost. These are the mobile equivalents of controls you applied to laptops earlier.
4. **Containerize work data.** Configure application management so that work applications and their data are separated from personal apps, a managed container. Add a piece of "work data" into the work container and some "personal data" outside it. The goal is to protect and control the work side without reaching into the personal side. This is how you secure data on a device you do not own.
5. **Set data-protection rules.** Within the work container, apply rules: prevent work data from being copied into personal apps, require the container to be encrypted, require authentication to open it. Test that moving work data to a personal app is blocked.
6. **Demonstrate the selective remote wipe. This is the key moment.** From the management platform, issue a remote wipe scoped to work data only. Confirm that the work container and its data are removed while the personal content remains intact. This is the control that makes mobile management acceptable to users: lose the device, or leave the company, and the work data goes without the personal photos going with it.
7. **Record the privacy balance.** Note what the management platform could see about the device and what it deliberately could not. Write down where you think the honest line is between managing the work data and respecting the person who owns the device.

Lab worksheet

Enrollment: Platform used: ____ What it can see: ____ What it cannot: ____

Baseline controls applied:

Control	Applied?
Device encryption confirmed	
Passcode/biometric required	

Containerization: Work data separated from personal? ____ Copy work-to-personal blocked? ____

Selective wipe:

After work-only wipe	Present or removed?
Work container data	
Personal data	

Privacy line, where you would draw it: ____

Expected outcomes

A managed mobile endpoint where work data is contained, protected, and selectively removable without touching personal content. The defining result is the selective wipe: work data gone, personal data intact. That single capability is what lets mobile management work in the real world, because it protects the organization's data without asking users to surrender control of their own device, and designing around that privacy tension is the whole art of mobile security.

Discussion questions

1. The selective wipe removed work data but not personal data. Why is that distinction so important for bring-your-own-device programs, and what happens to a program that can only wipe the entire device?
2. You noted what the management platform could and could not see. Where is the line between legitimate management and overreach, and how does crossing it lead to the "management users resented and evaded" failure?
3. A mobile endpoint is far more likely to be lost than a desktop. Which of the controls you applied matters most for a lost device, and which matters most for a compromised one, and are they the same?

Grading rubric

Criterion	Points
Device enrolled in management	20
Baseline mobile controls applied	15
Work data containerized and separated from personal	25
Data-protection rules applied and tested	15
Selective work-only wipe demonstrated successfully	20
Privacy balance reasoned through	5
Total	**100**

Lab 23: Provision and Secure a Cloud-Managed Endpoint

Companion to Video 23: Cloud-Managed and Modern Endpoints

Lab objectives

By the end of this lab you will be able to:

- Provision and secure a cloud-managed endpoint from scratch, without ever touching a corporate network.
- Tie the endpoint's trust to identity rather than to network location.
- Confirm the endpoint is protected in the always-connected, never-perimetered model.

Prerequisites

- A fresh endpoint VM (Windows 10/11) to enroll as if it were a brand-new device.
- Access to a cloud endpoint management / identity platform with a trial or free tier (the major cloud identity and device management platforms offer developer or trial tenants suitable for a single-device lab).
- Snapshot before starting.

Scenario

The modern endpoint is managed from anywhere, not from a corporate network it may never touch. Its security is tied to the identity using it rather than to where it sits. This lab provisions such a device the modern way, secure from first boot, configured over the internet, trusted because of who signs in rather than because of what network it is on, and confronts what endpoint security means when there is no perimeter to hide behind.

Step-by-step instructions

1. **Snapshot** the fresh VM as `pre-cloud-lab23`. Treat it as a device straight out of the box.
2. **Provision from the cloud.** Enroll the device into the cloud management platform during initial setup, so it is configured from the cloud before it is ever handled on a corporate network. Observe that the device becomes managed and policy-controlled over the internet alone. This is the provisioning model: secure from first boot, no physical handling required.
3. **Tie trust to identity.** Configure the endpoint so that access to resources depends on the identity signing in and the device's compliance state, not on being inside a network. Sign in as a managed identity and observe that the device's trust flows from that identity. This is the identity-centric model.
4. **Apply cloud-delivered policy.** Push your security baseline (encryption, endpoint protection, firewall, the controls from earlier labs) to the device from the cloud platform. Confirm it applied. The device is now hardened without ever having been on a corporate LAN.
5. **Confirm protection without a perimeter.** Verify the endpoint is protected and managed while sitting on an ordinary internet connection with no corporate network, no VPN-to-be-trusted, nothing. Its security does not depend on a network edge, because there is not one. Record how its protection is maintained.
6. **Test the identity-compromise angle.** Consider the modern failure mode: because trust flows from identity, compromising the identity can hand an attacker the endpoint. Confirm that multi-factor authentication and conditional access (from Lab 21's thinking) are protecting the identity, since the identity is now the keys to the device.
7. **Check for the silent-drift failure.** Confirm the cloud platform reports the device's compliance continuously, so a cloud-managed device that falls out of policy is noticed. The modern failure the course names is exactly the cloud-managed device that quietly drifted out of policy with no one watching.

Lab worksheet

Provisioning: Device enrolled from the cloud before any corporate network? ____ Managed over the internet alone? ____

Identity-centric trust: Resource access tied to identity + compliance? ____ MFA on the identity? ____

Cloud-delivered baseline:

Control	Pushed from cloud and applied?
Encryption	
Endpoint protection	
Firewall	

Perimeter-free protection: How protection is maintained without a network edge: ____

Continuous compliance reporting present? ____

Expected outcomes

A device that was provisioned, hardened, and made trustworthy entirely from the cloud, with its security anchored to identity and device health rather than to network location. The shift to internalize is that "inside the network" no longer means "trusted", a modern endpoint carries its security with it, tied to who is using it, and the network it happens to be on is irrelevant to whether it is trusted.

Discussion questions

1. The device was never on a corporate network, yet it is fully managed and hardened. What does that do to the old idea of a trusted internal network, and what replaces it?
2. Trust now flows from identity. The course warns of "the identity compromise that gave the attacker the endpoint." Why does the identity-centric model make protecting the identity, with MFA and conditional access, even more critical than before?
3. Endpoint, identity, and network security are converging in this model. Based on this lab, where do you see those three domains merging, and what does that convergence mean for how a defender should think about the endpoint?

Grading rubric

Criterion	Points
Device provisioned and managed from the cloud without a corporate network	25
Trust tied to identity and device compliance	20
Security baseline delivered from the cloud and applied	20
Protection maintained without a network perimeter, confirmed	15
Identity protected with MFA/conditional access	10
Continuous compliance reporting confirmed	10
Total	**100**

Lab 24: Defend a Server and an Unmanageable Endpoint

Companion to Video 24: Servers, IoT, and Non-Traditional Endpoints

Lab objectives

By the end of this lab you will be able to:

- Harden a lab server endpoint appropriately, recognizing how it differs from a user workstation.
- Design segmentation and compensating controls for an endpoint that cannot run a security agent.
- Reason about the range of non-traditional endpoints, servers, IoT, and operational technology, that strain the standard endpoint model.

Prerequisites

- WIN-SERVER (or a Linux server VM) as the server endpoint to harden.
- An "unmanageable" endpoint to design around: use a real IoT device from your Lab 1 inventory as the subject of the design exercise, or model one (a networked camera or sensor) that cannot run an agent.
- Snapshot before starting.

Scenario

Not every endpoint is a laptop, and the standard model strains at the edges. Servers are the highest-value endpoints and are defended differently from user devices. IoT and operational-technology devices often cannot be hardened, patched, or given an agent at all. This lab does both halves: it hardens a server for what a server is, and it designs protection for a device you cannot touch directly, because the device you cannot secure still has to be defended somehow.

Step-by-step instructions

Part A: the server endpoint

1. **Snapshot** WIN-SERVER.
2. **Harden for a server, not a laptop.** Apply a server-appropriate baseline. Note the differences from the workstation hardening in Lab 6: a server runs specific services for specific clients, so hardening is about minimizing to exactly those services and roles, locking down who can administer it, and heavy monitoring because of its value. Record three ways server hardening differs from workstation hardening.
3. **Monitor the server more closely.** Because it is a high-value endpoint, raise the telemetry and alerting on it above what you set for workstations. A server compromise matters more, so it earns more scrutiny. Configure that heightened monitoring.
4. **Confirm the server still serves.** As with all hardening, verify the server still does its job, clients can still reach the services they are supposed to. A server hardened into uselessness gets rolled back like any other over-tightened control.

Part B: the unmanageable endpoint

1. **Characterize the device.** Take your chosen IoT/OT device. Document what you cannot do to it: cannot install an agent, cannot patch, cannot change its configuration meaningfully, cannot log in and harden it. This is the reality of a large and growing class of endpoints.
2. **Design segmentation.** Since you cannot secure the device, protect the network from it and it from the network. Design a segment that isolates the device, allowing it only the connections it genuinely needs and nothing else. Draw the segment and its allowed flows. This is the primary defense for endpoints you cannot secure directly.
3. **Design compensating controls.** Around the segment, specify the compensating controls: monitor the device's traffic for anomalies (since you cannot monitor the device itself, watch what it does on the network), tightly control what can reach it and what it can reach, and plan what you would do if it were compromised, given that you cannot clean it.

4. **Address the operational-technology wrinkle.** For a device in the operational-technology class (industrial, where availability outweighs almost everything and patching can be dangerous), note how the priorities invert: you cannot take the "just patch it" or "just isolate it offline" actions you would take elsewhere, because keeping it running safely comes first. Record how that changes your design.

Lab worksheet

Part A, server hardening:

Three ways server hardening differed from workstation hardening: ____

Heightened monitoring configured: ____ Server still serves its clients? ____

Part B, unmanageable endpoint:

What you cannot do to the device	Confirmed
Install an agent	
Patch it	
Harden its config	

Segmentation design (attach drawing): Allowed flows in/out: ____

Compensating controls: ____

Operational-technology consideration: How availability-first priorities change the design: ____

Expected outcomes

A server hardened appropriately for its role and still serving its clients, plus a documented segmentation-and-compensating-control design for a device you cannot secure directly. The two halves make the same point from opposite ends: the endpoint model must flex to fit the endpoint. A server is not a laptop, an IoT sensor is not either, and defending them means adapting the model rather than forcing every device through the laptop-shaped controls from Part II.

Discussion questions

1. You listed three ways server hardening differs from workstation hardening. Which difference matters most, and why is "defend the server like a laptop" one of the failures the course names?
2. For the unmanageable device, your primary control was segmentation, not anything on the device. What does that tell you about where to invest when a whole class of endpoints cannot be secured directly?
3. Operational-technology devices invert the usual priorities, availability over almost everything. How does that change the "assume breach and contain" mindset from Lab 4, and what would it mean to contain a device you are not allowed to take offline?

Grading rubric

Criterion	Points
Server hardened appropriately with differences from workstation noted	20
Heightened server monitoring configured	10
Server confirmed still serving its clients	10
Unmanageable device characterized honestly	15
Segmentation designed with justified allowed flows	25
Compensating controls and OT consideration documented	20
Total	**100**

Part V: Operations, Program, and the Road Ahead

Lab 25: Route Endpoint Telemetry into Operations and Automate a Response

Companion to Video 25: The Endpoint in Security Operations

Lab objectives

By the end of this lab you will be able to:

- Route your lab endpoint telemetry into a central monitoring point.
- Correlate endpoint signal with identity signal to see a fuller picture than either gives alone.
- Build an automated isolation response that acts at machine speed.

Prerequisites

- The managed, telemetry-emitting fleet from Parts III and IV.
- MONITOR serving as the central monitoring point; if you have a lab SIEM or log-analytics tool, use it here.
- The isolation capability from Lab 10.
- Snapshot before starting.

Scenario

The endpoint is the richest signal source a security operation has, and this lab connects it to the operation it serves. You will route endpoint telemetry into one place, correlate it with identity so that an endpoint event and a login event tell a combined story, and then automate the response so that a clear detection isolates a machine without waiting for a human. The failures to avoid are the signal that never arrived, the alert flood that blinded the analysts, and the response that could not be automated.

Step-by-step instructions

1. **Snapshot** the fleet.
2. **Centralize the telemetry.** Confirm every endpoint's telemetry is flowing into your central monitoring point, and that an analyst sitting at MONITOR could see activity across the whole fleet from one screen. Fix any endpoint whose signal is not arriving, the signal that never reached operations is a blind spot nobody chose.
3. **Add identity signal.** Bring authentication events into the same place: logins, failures, privilege use (from your Active Directory or identity source). Now endpoint events and identity events live together.
4. **Correlate the two domains.** Construct a correlated view that combines them. For example: a suspicious process on WIN-ENDPOINT (endpoint signal) plus an unusual login to that machine moments before (identity signal) is a stronger, clearer story than either alone. Run a small attack that produces both kinds of signal and build the correlation that ties them into one picture. The fullest picture combines endpoint signal with identity (and network).
5. **Tune for signal quality.** Look at the alert volume reaching your monitoring point. Tune it: suppress or lower the noisiest low-value alerts so the high-value ones are not buried. An operations center that is drowning sees nothing, and over-alerting is itself a failure. Record what you tuned and why.
6. **Automate an isolation response.** Build an automated response: when a high-confidence detection fires (for example, the honey-credential alert from Lab 17, which is near-certain to be real), automatically trigger the endpoint isolation from Lab 10, no human in the loop for that first containment. Machine-speed response matters because an attacker moves faster than a human analyst can click.

7. **Test the automation.** Trigger the high-confidence detection with a simulated attack and confirm the endpoint is automatically isolated within seconds. Record the time from detection to isolation. Then confirm you can review and reverse the automated action, automation you cannot undo is its own hazard.
8. **Consider the analyst.** Note what an analyst would need to work effectively here: the correlated view, the ability to pivot from an alert to the endpoint's full telemetry, and the control to isolate or release a machine. Write down whether your setup gives them that.

Lab worksheet

Centralization: All endpoints' telemetry arriving centrally? ____ Any gaps found and fixed: ____

Identity signal added? ____

Correlation built: Endpoint + identity scenario correlated: ____

Tuning: Alerts suppressed/lowered and why: ____

Automated response:

Trigger	Automated action	Time detection to action	Reversible?

Analyst needs met? ____

Expected outcomes

A small but real security operation: endpoint and identity telemetry in one place, a correlation that tells a combined story, tuning that keeps the signal above the noise, and an automated isolation that contains a high-confidence threat in seconds. The realization to carry into the program labs is that the endpoint is not an island, its value multiplies when its signal is correlated with the other domains and acted on fast, and that is what "security operations" actually means from the endpoint's point of view.

Discussion questions

1. The correlated endpoint-plus-identity view was stronger than either signal alone. Give another pairing of signals that would catch something neither would catch by itself.
2. You automated isolation only for a high-confidence trigger. Why is confidence the deciding factor for what you automate, and what would go wrong if you auto-isolated on a noisy behavioral alert instead?
3. Tuning reduced the alert volume. There is a risk in suppressing alerts, you might suppress a real one. How do you tune aggressively enough to keep analysts functional without tuning away the thing that mattered?

Grading rubric

Criterion	Points
All endpoint telemetry centralized, gaps fixed	20
Identity signal integrated	15
Endpoint-and-identity correlation built and demonstrated	20
Alert tuning performed with justification	15
Automated isolation built, tested, timed, and reversible	25
Analyst needs considered	5
Total	**100**

Lab 26: Design an Endpoint Security Awareness Approach

Companion to Video 26: The Human Factor and Security Awareness

Lab objectives

By the end of this lab you will be able to:

- Design an endpoint security awareness approach that makes users allies rather than adversaries.
- Specify how a user would report a mistake they just made, and why the culture around that matters.
- Account for the insider dimension without turning every user into a suspect.

Prerequisites

- No machines required. This is a design lab, producing a written approach.
- Your experience from the earlier labs, especially the usability tensions in Labs 4, 6, 8, and 22.

Scenario

The most hardened endpoint is compromised the moment its user is deceived, which makes the person at the keyboard both the greatest weakness and, when trained and engaged, one of the best detection mechanisms the endpoint has. This lab designs the human side of endpoint security. The target is not a user who passes a quiz; it is a user who is an ally, who reports the phishing click they just made instead of hiding it, because hiding it is what turns a small mistake into a breach.

Step-by-step instructions

1. **Define the user as attack surface.** List the ways attackers target the person rather than the machine on your endpoints: phishing, social engineering, manipulation, malicious attachments and links. These are the human entry points no technical control fully closes.
2. **Define the user as sensor.** Now flip it: list the ways a trained, engaged user is a detection mechanism. A user who notices and reports a suspicious email is catching attacks your tools might miss. Design for this, not just against the weakness.
3. **Design awareness that changes behavior.** Specify a training approach that actually changes behavior rather than checking a box. Be concrete about what makes the difference: relevance to the user's real work, realistic practice (such as simulated phishing used to teach, not to punish), short and frequent over long and annual, and tied to the specific endpoint risks in your environment. State how you would measure whether behavior actually changed.
4. **Build the usability partnership.** Connect this back to the course's recurring theme: security that respects the user's work earns cooperation; security that fights the user gets defeated. Specify how your awareness approach treats users as partners, and name one control from an earlier lab you would adjust to reduce the friction that drives users to work around security.
5. **Design the reporting path. This is the core.** Specify exactly how a user reports a mistake, a clicked link, a lost device, an entered password on a fake page, in the simplest possible way, and specify the cultural commitment that makes them willing to: no blame for honest reports, fast and grateful response, and the message that reporting a mistake early is a win, not a confession. The user who hides their mistake is the failure this is designed to prevent.
6. **Address the insider dimension carefully.** Acknowledge that some endpoint threats come from trusted, authorized users. Specify how you would watch for that, through the behavioral and deception signals from earlier labs, without poisoning the ally relationship you just built by treating every user as a suspect. This balance is delicate and worth stating explicitly.
7. **Write the one-page approach.** Pull it together into a one-page endpoint security awareness approach that a real small organization could adopt.

Lab worksheet

User as attack surface (entry points): ____

User as sensor (detection contributions): ____

Awareness design: What makes it change behavior: ____ How you would measure behavior change: ____

Usability partnership: One earlier control you would adjust to reduce friction: ____

Reporting path: How a user reports a mistake (the mechanics): ____ The cultural commitments that make them willing: ____

Insider dimension: How you watch for it without treating everyone as a suspect: ____

One-page approach attached? ____

Expected outcomes

A written awareness approach that treats users as allies and sensors, not just as a liability, and that makes reporting a mistake easy and safe enough that people actually do it. The piece that distinguishes a good answer is the reporting culture: technical controls can be specified in an afternoon, but an environment where a user comfortably says "I think I just clicked something I shouldn't have" is worth more than most of them, because it converts the inevitable human error into an early warning instead of a hidden breach.

Discussion questions

1. The same user is both the weakest surface and a strong sensor. Which of those two framings does most security training emphasize, and what is lost by emphasizing the weakness over the sensor?
2. Your reporting path depends on a no-blame culture. What specifically goes wrong, in security terms, the first time a user is punished for an honest report?
3. The insider dimension pulls against the ally relationship. Is it possible to watch for insider threats and build a trusting reporting culture at the same time? Argue your position.

Grading rubric

Criterion	Points
User as attack surface and as sensor both developed	20
Awareness design is concrete and tied to behavior change with a measure	20
Usability partnership addressed with a specific control adjustment	15
Reporting path designed, mechanics and culture both specified	30
Insider dimension handled without poisoning the ally relationship	10
Coherent one-page approach produced	5
Total	**100**

Lab 27: Design an Endpoint Security Program

Companion to Video 27: Building and Running an Endpoint Security Program

Lab objectives

By the end of this lab you will be able to:

- Assemble the controls from the whole course into a coherent, layered endpoint security program rather than a pile of tools.
- Define metrics that measure real risk reduction, and identify the vanity metrics that mislead.
- Produce an endpoint security program design with a layered architecture, metrics, and a maturity roadmap, the takeaway artifact for this video.

Prerequisites

- Everything you built in Labs 1 through 26. This lab is where it becomes a program.
- No new machines; this is a synthesis and design lab producing a document.

Scenario

The controls in this course only work as a managed, ongoing whole. A pile of good tools that nobody runs as a program is one of the central failures the course names. This lab turns your scattered labs into a designed program: the strategy, roles, tooling, policy, and operations that hold the controls together, measured by metrics that reflect actual risk reduction, with an honest assessment of where the program stands and where it needs to go.

Step-by-step instructions

1. **List the program elements.** Lay out the five elements: strategy (what you are protecting and from what), roles (who does what), tooling (the controls and platforms you built), policy (from Lab 21), and operations (from Lab 25). A program is all five working together, not any one of them.
2. **Draw the layered-defense architecture.** Assemble your controls into a single layered picture: hardening and attack-surface reduction (Part II) as the foundation, prevention/detection/response (Part III) on top, fleet management/patching/policy (Part IV) making it consistent, and operations/human factor (Part V) completing it. Each layer catches what the last missed. This diagram is the heart of the artifact.
3. **Assess maturity honestly.** Rate where your program stands on a simple maturity scale (for example: ad hoc, repeatable, defined, managed, optimizing) across a few dimensions, prevention, detection, response, management, human factor. Be honest; the point of a maturity model is to show the gap, not to flatter.
4. **Choose metrics that matter.** Define a small set of metrics that measure real risk reduction: coverage (what fraction of endpoints are managed, hardened, reporting), time to detect, time to respond, and real risk reduction indicators. These answer "are we actually safer?"
5. **Name the vanity metrics to avoid.** Explicitly list the misleading metrics: counts of blocked threats (which say nothing about what got through), number of tools owned, training-completion percentages divorced from behavior change. Note next to each why it misleads. The metric that measures the wrong thing is a named failure.
6. **State the economics.** Write how you would justify the program as risk reduction, not as a cost. Tie spending to the risk it reduces, so the program can demonstrate its value, the program that could not demonstrate its value is the last failure in this video.
7. **Build the maturity roadmap.** Lay out a path from where the program is now to where it should be: the next few improvements in priority order, each tied to a gap from your maturity assessment.
8. **Assemble the artifact.** Produce the endpoint security program design: layered architecture diagram, metrics (real and vanity-avoided), maturity assessment, and roadmap, as one coherent document.

Lab worksheet

Program elements present: Strategy ____ Roles ____ Tooling ____ Policy ____ Operations ____

Layered architecture diagram attached? ____

Maturity assessment:

Dimension	Current level	Target level
Prevention		
Detection		
Response		
Management		
Human factor		

Metrics that matter: ____

Vanity metrics avoided (and why each misleads): ____

Economics / value justification: ____

Roadmap (next improvements in priority order): ____

Expected outcomes

A single document that turns twenty-six labs' worth of controls into a program someone could actually run: a layered architecture, an honest maturity picture, metrics that answer whether you are safer, and a roadmap. The shift this lab forces is from "I configured a lot of controls" to "I run a program that manages those controls as a whole," which is the difference between endpoint security as a project and endpoint security as a discipline.

Discussion questions

1. The course says a pile of tools is not a program. From your own earlier labs, which controls would be nearly worthless without the program elements (roles, operations, policy) wrapped around them?
2. You separated metrics that matter from vanity metrics. Why are blocked-threat counts so popular despite saying nothing about what got through, and what pressure pushes organizations toward vanity metrics?
3. Your maturity assessment showed gaps. If you could only close one gap in the next quarter, which would it be, and how did your metrics inform that choice?

Grading rubric

Criterion	Points
All five program elements laid out	15
Coherent layered-defense architecture diagram	25
Honest maturity assessment across dimensions	15
Metrics that measure real risk reduction defined	20
Vanity metrics identified with reasons they mislead	15
Maturity roadmap produced	10
Total	**100**

Lab 28: Build an Endpoint Tool Evaluation Rubric

Companion to Video 28: The Endpoint Security Ecosystem and Tool Selection

Lab objectives

By the end of this lab you will be able to:

- Make sense of the crowded endpoint security market and its overlapping categories.
- Evaluate an endpoint security tool by how it serves your actual defense rather than by its marketing.
- Produce an endpoint security tool evaluation rubric mapping capabilities to the defenses each is meant to provide, the takeaway artifact for this video.

Prerequisites

- Your program design from Lab 27, which defines the defenses you are actually trying to provide.
- No machines required; this is a research-and-design lab producing a rubric.

Scenario

The endpoint security market is a wall of overlapping categories and acronyms, antivirus, endpoint protection platform, endpoint detection and response, extended detection and response, and more, and the overlap is itself a barrier to choosing well. This lab builds the defense against being ruled by the market: a rubric that evaluates any tool by the defenses it actually provides for your environment, so you buy capability you need rather than marketing you were sold.

Step-by-step instructions

1. **Map the categories.** Lay out the main tool categories and, in plain language, what each actually does: antivirus / endpoint protection (prevention of known-bad), endpoint detection and response (behavior recording and response), and the extended/unified platforms (correlating endpoint with identity, network, cloud). Cut through the acronyms to the function. Note where the categories overlap, because the overlap is where the confusion lives.
2. **Note the convergence.** Record that these categories are merging, endpoint, identity, network, and cloud detection are converging into unified platforms. A tool bought today should be judged partly on how it fits that convergence.
3. **Derive your evaluation criteria from your defenses.** This is the key move. Take the defenses from your Lab 27 program (hardening support, prevention, detection coverage, response capability, fleet management, telemetry quality, integration) and turn each into an evaluation criterion. You are evaluating tools against the defense you need, not against a generic feature list.
4. **Add the practical criteria.** Include criteria that marketing hides: integration with what you already run (the value is in tools working together), the management burden, whether it can be run by a team your size or needs a managed service, and total cost against risk reduced.
5. **Build the rubric.** Assemble a scored rubric: each capability/criterion as a row, with a way to rate a given tool against it (for example, 0 to 3), weighted by how much that capability matters to your environment. The rubric maps capabilities to the defenses they provide, so a tool's score reflects how well it serves your actual defense.
6. **Add the build-versus-buy-versus-managed question.** Include a section that forces the decision: for this capability, do you build it (as you did with open-source tools in the labs), buy a product, or have it run as a managed service? The right answer depends on your team's size and maturity, from Lab 27.
7. **Test the rubric.** Apply it to two tools you used in the labs (for example, the open-source telemetry stack from Lab 13 versus a hypothetical commercial EDR) and confirm the rubric produces a defensible comparison rather than a popularity contest.

Lab worksheet

Category map:

Category	What it actually does	Overlaps with
Antivirus / EPP		
EDR		
Extended/unified platform		

Evaluation criteria derived from your defenses: ____

Practical criteria added: ____

Rubric built (attach): Capabilities as rows, weighted, scorable? ____

Build / buy / managed section included? ____

Rubric tested on two tools: Result: ____

Expected outcomes

A reusable rubric that scores any endpoint security tool by the defenses it provides for your environment and by how it integrates with what you already run, with an explicit build-versus-buy-versus-managed decision built in. Applied to two real tools, it should produce a comparison you could defend to someone holding a budget. The habit it builds is reading every product claim through the question "which defense does this actually provide for me?", which is the only reliable guard against buying marketing.

Discussion questions

1. The market's overlapping categories are themselves a barrier. How does deriving your criteria from your own defenses, rather than from vendors' category labels, change what you end up buying?
2. The course warns of "the products that did not integrate." Why is integration so often undervalued at purchase time and so painful afterward, and how does your rubric force it into the decision?
3. Build, buy, or managed, when is the open-source build approach you used in the labs the right call for a real organization, and when does it become a false economy?

Grading rubric

Criterion	Points
Tool categories mapped to real functions, overlaps noted	20
Convergence toward unified platforms acknowledged	10
Evaluation criteria derived from actual defenses, not a generic list	25
Practical criteria (integration, burden, cost) included	15
Scored, weighted rubric built	20
Build/buy/managed decision included and rubric tested	10
Total	**100**

Lab 29: Build a Fully Defended Endpoint and Walk an Attack Through It

Companion to Video 29: The Complete Endpoint Defense End to End

Lab objectives

By the end of this lab you will be able to:

- Build a fully defended endpoint from bare metal through every layer the course taught.
- Run a multi-stage attack against it and watch each layer catch what the last one missed.
- Articulate how every part of the course connects in a single coherent defense.

Prerequisites

- A fresh endpoint VM to build from scratch.
- Every skill and control from Labs 6 through 25.
- ATTACK-HOST and your telemetry/operations setup.
- This is the integration capstone of the hands-on course; budget real time for it.

Scenario

This is where the whole course comes together on one machine. You start from a bare endpoint and build the full layered defense outward, hardening first, then prevention, detection, and response, then the management and operations that keep it consistent, then you run a real multi-stage attack and watch the layers catch it in sequence. Seeing each layer catch what the previous one missed is the entire thesis of the course made visible in a single exercise.

Step-by-step instructions

1. **Start bare.** Build a fresh endpoint VM and snapshot it as `bare-lab29`. This is the starting point: a machine with nothing but a default OS install.
2. **Layer 1, harden (from Part II).** Apply the hardening baseline (Lab 6), remove admin rights and fix local credentials (Lab 7), enable application control (Lab 8), encrypt the disk (Lab 9), configure the host firewall with outbound control (Lab 10), and enable boot/physical protections (Lab 11). Snapshot as `hardened-lab29`.
3. **Layer 2, prevent and detect (from Part III).** Deploy endpoint protection (Lab 12), deploy detection/telemetry (Lab 13), plant deception (Lab 17), and enable tamper protection and off-machine log forwarding (Lab 18). Snapshot as `defended-lab29`.
4. **Layer 3, manage (from Part IV).** Bring the endpoint under central management, enforce the baseline, confirm it is patched, and place it under policy with conditional access (Labs 19 through 21).
5. **Layer 4, operate (from Part V).** Confirm its telemetry flows into your central monitoring, correlated with identity, with the automated isolation response armed (Lab 25). The endpoint is now defended in depth, managed, and operated.
6. **Snapshot the finished state** as `complete-lab29`. Then design a multi-stage attack that touches several layers: delivery, execution, privilege, persistence, credential access, and attempted command-and-control / lateral movement.
7. **Run the attack and watch the layers.** Execute the attack stage by stage. At each stage, record which layer caught it, prevented it, detected it, or forced the attacker to change approach:
   - Did hardening/application control block the initial execution?
   - If the attacker went fileless to evade that, did detection catch the behavior?
   - Did removed admin rights stop privilege escalation?
   - Did outbound control block command-and-control?
   - Did the honey objects fire a high-fidelity alert?

- Did the automated response isolate the machine?

Build the sequence: for each attacker move, the layer that answered it.

1. **Tell the story.** Write the narrative of the attack against your fully defended endpoint: where each layer caught what the last missed, and whether the attacker ever reached their goal. This narrative is the deliverable.
2. **Restore.**

Lab worksheet

Layers built (snapshot after each):

Layer	Controls applied	Snapshot taken
Harden		
Prevent/detect		
Manage		
Operate		

The attack, stage by stage:

Attack stage	What the attacker tried	Layer that answered	Prevented / detected / forced a change
Delivery			
Execution			
Privilege			
Persistence			
Credential access			
Command-and-control / lateral			

Did the attacker reach their goal? ____

Attack narrative attached? ____

Expected outcomes

A single endpoint carrying every control the course taught, and a stage-by-stage record of a multi-stage attack meeting layer after layer. The expected and instructive result is that no single layer stops everything, but the layers together stop the attack or at minimum detect and contain it before the goal, with at least one stage where a layer catches exactly what the previous one missed. That moment, prevention misses, detection catches, is the course's whole argument, and now you have produced it yourself.

Discussion questions

1. Identify the single clearest moment where one layer caught what another missed. What would have happened with only the first layer in place?
2. Did any attack stage get through every layer? If so, what layer is missing from your defense; if not, which attacker change-of-approach came closest to a gap?
3. You built this on one machine. Everything in Part IV exists so this same defense can be consistent across thousands. Which layers were only possible because of the management and operations work, and which could a lone defender build on a single machine?

Grading rubric

Criterion	Points
Endpoint built from bare metal through all four layers	30
Snapshots taken at each layer	5
Multi-stage attack designed to touch several layers	15
Each stage mapped to the layer that answered it	30
At least one clear "next layer caught the miss" moment identified	10
Coherent end-to-end attack narrative produced	10
Total	**100**

Lab 30: Audit Your Defense Against the Failure Catalog

Companion to Video 30: When Endpoint Security Fails

Lab objectives

By the end of this lab you will be able to:

- Audit your lab endpoint defense against the course's catalog of failure modes.
- Tell a hard threat apart from a broken defense.
- Remediate every gap the audit finds, designing the failures out.

Prerequisites

- The fully defended endpoint and fleet from Lab 29.
- The failure modes named throughout the course, gathered here into an audit checklist.
- Snapshot before starting.

Scenario

The course gathers the discipline's failure modes in one place so you can design them out rather than rediscover them in production. A security failure is often not a clever threat but a broken defense: a control that drifted, an alert nobody watched, a user who hid a mistake. This lab turns the failure catalog into an audit, runs it against the defense you built, and fixes what it finds. The pattern underneath every failure is the same, and so is the fix: layered defense, applied honestly.

Step-by-step instructions

1. **Snapshot** the fleet.
2. **Run the hardening-failure audit.** Check for: the unpatched endpoint, the user running as admin, the baseline that drifted. On your own machines, verify patch currency, confirm no user has unnecessary admin rights, and run a drift check against your baseline. Record pass/fail for each and fix any failure.
3. **Run the prevention-failure audit.** Check for: a novel/fileless threat that evades signatures (confirm you have behavioral detection behind antivirus), a control trivially bypassed (confirm layering behind each single control), and the security tool the malware disabled (confirm tamper protection is on). Record and fix.
4. **Run the detection-and-response audit.** Check for: the alert lost in the noise (confirm your tuning from Lab 25 keeps high-value alerts visible), the compromise found too late (confirm your telemetry would surface the Lab 5 techniques), and the response that did not scale (confirm your isolation and response work from the central platform). Record and fix.
5. **Run the management-failure audit.** Check for: the unmanaged endpoint nobody knew about (reconcile your Lab 1 inventory against your Lab 19 managed fleet, find anything unmanaged), configuration drift (re-run the drift check), and the agent that silently stopped reporting (confirm you would detect a silent agent). Record and fix.
6. **Run the human-failure audit.** Check for: training that changed nothing, the hidden mistake, the security so painful it was defeated, and the unwatched insider. Audit your Lab 26 approach against each. Record and note remediations (these are design fixes, not machine settings).
7. **Run the program-failure audit.** Check for: the pile of tools that was not a program, the single control trusted to hold, and the value never demonstrated. Audit your Lab 27 program against each. Record and fix.
8. **Separate threats from broken defenses.** For any gap you found, classify it: was it a genuinely hard threat, or a defense that was simply broken, drifted, disabled, or never watched? Most will be the latter. This classification is the lab's core insight, and it tells you where your effort actually needs to go.
9. **Confirm remediation.** Re-run the audit and confirm every gap is closed.

Lab worksheet

Failure category	Specific failure checked	Pass/Fail	Threat or broken defense?	Remediation
Hardening	Unpatched endpoint			
Hardening	User as admin			
Hardening	Baseline drift			
Prevention	Fileless evades signatures			
Prevention	Control trivially bypassed			
Prevention	Security tool disabled			
Detection	Alert lost in noise			
Detection	Found too late			
Detection	Response did not scale			
Management	Unmanaged endpoint			
Management	Config drift			
Management	Silent agent			
Human	Training changed nothing			
Human	Hidden mistake			
Human	Security defeated by friction			
Human	Unwatched insider			
Program	Tools, not a program			
Program	Single control trusted			
Program	Value not demonstrated			

How many gaps were broken defenses rather than hard threats? ____

Expected outcomes

A completed audit across all six failure categories, every gap classified as a hard threat or a broken defense, and every gap remediated. The number that matters is the one at the bottom: how many of your gaps were broken defenses rather than hard threats. For almost everyone it is most of them, which is the lab's lesson, the discipline fails far more often from defenses that quietly broke than from attackers who were brilliant, and that is good news, because broken defenses are the thing you can design out.

Discussion questions

1. Most of your gaps were probably broken defenses, not hard threats. What does that imply about where a defender's time is best spent, cleverness against sophisticated attackers, or diligence against your own controls breaking?
2. The same pattern, layered defense applied honestly, guards against failures in every category. Pick two very different failures from your audit and show how that one pattern addresses both.
3. A drifted baseline and a disabled agent look, from a distance, like a healthy machine. What does that tell you about the difference between a control being present and a control being verified?

Grading rubric

Criterion	Points
All six failure categories audited	30
Each specific failure checked against the real defense	25
Gaps classified as threat vs broken defense	15
Every gap remediated	20
Remediation confirmed on re-audit	10
Total	**100**

Lab 31: Diagnose Staged Failures Across the Defense

Companion to Video 31: Troubleshooting Endpoint Security

Lab objectives

By the end of this lab you will be able to:

- Diagnose staged failures across the endpoint defense, from a disabled agent to a detection gap to a false positive breaking legitimate work.
- Tell a real compromise apart from a broken control.
- Build a personal endpoint security troubleshooting checklist spanning hardening, controls, management, and the user.

Prerequisites

- The defended fleet from Lab 29/30.
- A partner to stage faults, or stage them yourself and diagnose after a delay so you approach them fresh.
- Snapshot before starting.

Scenario

Endpoint troubleshooting is its own discipline because a problem could live anywhere, in the hardening, the controls, the management layer, or the user. The hardest and most important judgment is distinguishing a real attack from a control that simply broke, because the two can look alike and the responses are opposite. This lab stages a set of realistic faults across the whole defense and has you diagnose each, then distills the experience into a checklist you keep.

Step-by-step instructions

1. **Snapshot** the fleet. Have your partner (or your past self) stage a set of faults without telling you which is which. A good set:
   - A telemetry/EDR agent stopped on one endpoint.
   - A policy that did not apply to one machine.
   - A detection gap, an attack technique that is not being caught.
   - A false positive, a control blocking a piece of legitimate work.
   - A performance problem, security tooling slowing a machine noticeably.
   - A drifted or unmanaged machine in the fleet.
   - One actual (simulated) compromise hidden among the broken controls.
1. **Diagnose the control failures.** For the stopped agent, the unapplied policy, and the disabled protection, work out what broke and why. Is the agent crashed, blocked, or uninstalled? Did the policy fail to apply because of connectivity, scope, or conflict? Record the diagnosis and the evidence for each.
2. **Diagnose the detection gap.** For the attack that is not being caught, determine why: is the telemetry not collecting the right data, is there no rule for it, or is the signal being tuned out? This is diagnosing an absence, which is harder than diagnosing an error. Record how you found it.
3. **Diagnose the false positive.** For the control blocking legitimate work, confirm it is a false positive and not a real block of something that should be blocked, then determine the tuning that fixes it without opening a hole. This is the usability-tension failure in diagnostic form.
4. **Diagnose the performance problem.** For the machine slowed by security tooling, identify which control is responsible and what the fix is, because a security tool that makes a machine unusable gets removed by users, and then you have no security at all.
5. **Diagnose the fleet problem.** For the drifted or unmanaged machine, find it from the central platform and determine how it got that way.

6. **Answer the central question for each: compromise or malfunction?** For every fault, and especially for the hidden real compromise, classify it: is this a broken control or an actual attack? State the evidence that distinguishes them. The agent that stopped could be a crash, or it could be malware disabling it, and telling which is the whole game. Find the one real compromise in the set.
7. **Build your checklist.** From the whole exercise, write a personal endpoint security troubleshooting checklist spanning hardening, controls, management, and the user, the ordered questions you would ask to localize any future endpoint problem.

Lab worksheet

Staged fault	Diagnosis	Evidence	Compromise or malfunction?
Stopped agent			
Unapplied policy			
Detection gap			
False positive			
Performance problem			
Drifted/unmanaged machine			
Hidden real compromise			

The one real compromise was: ____ How I distinguished it from the broken controls: ____

Personal troubleshooting checklist attached? ____

Expected outcomes

Every staged fault correctly diagnosed and, crucially, the one real compromise correctly separated from the broken controls around it. The checklist you build is the lasting artifact, a repeatable method for localizing a problem that could be anywhere in a layered defense. The judgment the lab trains, is this a broken control or an attack?, is the one that most often determines whether a real incident is caught early or dismissed as a glitch.

Discussion questions

1. The stopped agent could be a crash or malware disabling it. What evidence lets you tell the two apart quickly, and what is the cost of guessing wrong in each direction?
2. Diagnosing the detection gap meant diagnosing an absence of signal rather than an error. Why is "nothing happened when something should have" so much harder to notice and diagnose than an outright failure?
3. The false positive and the performance problem are both usability failures that lead to controls being removed. How does your troubleshooting checklist account for the fact that the most dangerous failure is sometimes the control that worked but annoyed people into disabling it?

Grading rubric

Criterion	Points
Control failures (agent, policy, protection) correctly diagnosed	20
Detection gap diagnosed as an absence of signal	15
False positive diagnosed and a sound tuning fix identified	15
Performance and fleet problems diagnosed	15
Each fault correctly classified as compromise or malfunction	20
Personal troubleshooting checklist produced	15
Total	**100**

Lab 32: Assess Your Skills and Write a Development Plan

Companion to Video 32: Building Skills and Where to Go Next

Lab objectives

By the end of this lab you will be able to:

- Assess your skills against the whole course honestly.
- Choose a specialization within endpoint security.
- Write a development plan, including a 90-day plan and an ongoing endpoint defense-and-attack practice habit.

Prerequisites

- Completion of the course and the preceding labs.
- Your worksheets from Labs 1 through 31, which are the evidence of what you can now do.

Scenario

The course ends; the field does not. This final lab turns everything you built into a forward path. The durable core, the harden-detect-respond-manage spine, is what ages well while specific tools come and go. You will assess where you stand against the whole course, choose a direction, and commit to a concrete plan, because the skills you do not keep using are the skills you lose, and a lab you keep attacking and defending is how you keep them.

Step-by-step instructions

1. **Assess against the course.** Go through the five parts and rate yourself honestly on each major capability: can you harden an endpoint, remove privilege, control execution, encrypt data, defend the network edge, secure boot? Can you deploy detection, hunt, investigate, respond, use deception, understand evasion? Can you manage a fleet, patch it, govern it with policy, handle mobile and cloud and non-traditional endpoints? Can you run operations, build a program, evaluate tools? Use your worksheets as evidence, not memory.
2. **Identify the durable core you built.** Name the parts of the harden-detect-respond-manage spine you are now solid on. These are the concepts that do not change, the foundation everything else attaches to.
3. **Survey the specializations.** Consider the main directions this course opens: endpoint detection and response engineering, threat hunting, endpoint forensics, and fleet/modern-management engineering. For each, note what it involves day to day and which of your labs you most enjoyed and did best, because that is a real signal about fit.
4. **Choose a specialization.** Pick one direction to go deeper. Write why, grounded in your self-assessment and which labs engaged you most.
5. **Map the certifications.** For your chosen direction, note which certifications signal relevant skill to employers and what each actually tests. Treat them as signals and structure, not as the learning itself.
6. **Commit to a practice habit.** Specify how you will keep a lab of endpoints to defend and attack going after the course. The lab you built in Lab 3 does not have to be torn down; keeping it, and periodically attacking and defending it, is how the skills stay sharp. State a realistic cadence.
7. **Plan how you will follow the field.** Identify the researchers, threat reporting, and communities you will track, and note where the field is heading, the convergence of endpoint, identity, and network defense, the impact of AI on both attack and defense, and the vanishing perimeter, so your learning points forward.
8. **Write the 90-day plan.** Produce a concrete 90-day plan to consolidate these skills and move into your chosen specialization: specific goals, what you will build or study each month, and the ongoing defend-and-attack practice folded in.

Lab worksheet

Self-assessment (rate each part, with a worksheet as evidence):

Part	Capabilities	Confidence (L/M/H)	Evidence (which lab)
I Foundations			
II Harden/prevent			
III Detect/respond			
IV Manage at scale			
V Operations/program			

Durable core I am solid on: ____

Specialization chosen: ____ Why (grounded in the assessment): ____

Relevant certifications and what they signal: ____

Ongoing lab practice cadence: ____

Field-following plan: ____

90-day plan attached? ____

Expected outcomes

An honest skills assessment backed by the evidence of your own completed labs, a chosen specialization that fits where you did your best work, and a concrete 90-day plan with an ongoing defend-and-attack practice built in. The point of ending here is momentum: you finish the course not with a certificate feeling but with a direction and a dated plan, and with a lab still standing that you can keep breaking and fixing, which is the only way the harden-detect-respond-manage spine stays a live skill rather than a memory.

Discussion questions

1. The harden-detect-respond-manage spine is the durable core. Looking back at the whole course, why do those four verbs survive changes in tools and threats, and what would it take for them to stop being the right frame?
2. You chose a specialization partly from which labs engaged you most. How much should enjoyment weigh against market demand in that choice, and how would you test whether your chosen direction actually suits you before committing years to it?
3. The field is converging, endpoint, identity, and network defense merging, with AI reshaping both attack and defense. How does that convergence change what "specializing in endpoint security" will mean five years from now, and how does your plan stay flexible to it?

Grading rubric

Criterion	Points
Honest self-assessment across all five parts, evidenced by worksheets	25
Durable core identified	10
Specializations surveyed and one chosen with grounded reasoning	20
Certifications mapped to the chosen direction	10
Ongoing defend-and-attack practice habit committed to	15
Concrete 90-day development plan produced	20
Total	**100**

Appendix A: Lab-to-Video Map

Lab	Video	Focus	Primary machine(s)
1	1	Inventory and classify endpoints	Real environment
2	2	Map an endpoint's attack surface	WIN-ENDPOINT
3	3	Build the lab	All
4	4	Defend-and-bypass on paper	None
5	5	Observe attack techniques safely	WIN-ENDPOINT, ATTACK-HOST
6	6	Apply a hardening benchmark	WIN-ENDPOINT
7	7	Remove admin, fix local credentials	WIN-ENDPOINT, WIN-SERVER
8	8	Application control and bypass	WIN-ENDPOINT
9	9	Disk encryption and its limits	WIN-ENDPOINT or LIN-ENDPOINT
10	10	Host firewall, outbound control	WIN-ENDPOINT, ATTACK-HOST
11	11	Secure boot, hardware protections	WIN-ENDPOINT
12	12	Endpoint protection and evasion	WIN-ENDPOINT
13	13	Deploy EDR, catch an attack	WIN-ENDPOINT, MONITOR
14	14	Hunt for persistence	WIN-ENDPOINT, LIN-ENDPOINT
15	15	Investigate a compromise	WIN-ENDPOINT
16	16	Full incident lifecycle	WIN-ENDPOINT
17	17	Deception and honey objects	WIN-ENDPOINT, MONITOR
18	18	Evade controls, add layers	WIN-ENDPOINT
19	19	Manage a fleet, catch drift	All
20	20	Scan, prioritize, patch	All
21	21	Policy and conditional access	All
22	22	Mobile device management	Mobile device/emulator
23	23	Cloud-managed endpoint	Fresh VM + cloud platform
24	24	Server and unmanageable endpoints	WIN-SERVER, IoT device
25	25	Operations and automated response	All, MONITOR
26	26	Security awareness design	None
27	27	Endpoint security program	None
28	28	Tool evaluation rubric	None
29	29	Full defense end to end	Fresh VM, all
30	30	Failure-catalog audit	All
31	31	Troubleshooting staged faults	All

Lab	Video	Focus	Primary machine(s)
32	32	Skills assessment and plan	None

Appendix B: Snapshot Discipline Quick Reference

Take a named snapshot before any lab that compromises or significantly changes a machine. Restore when you have recorded what you needed. Suggested names used in this guide:

- clean-baseline — a fresh, updated machine with nothing done to it yet.
- pre-<topic>-lab<N> — the state just before a specific lab breaks something (for example pre-forensics-lab15).
- hardened-lab29, defended-lab29, complete-lab29 — the layered build points in the end-to-end lab.

Never leave a knowingly compromised machine running on the lab network. Snapshots are not backups of your real work; they are the reset button that makes attacking your own machines safe.

Appendix C: A Note on Tools

This guide names tool categories and, where it helps, specific open-source tools (Sysmon, Autoruns, OpenVAS/Greenbone, AppLocker, BitLocker, LUKS, Ansible, Active Directory with Group Policy). These are chosen because they run in a home lab without licensing cost and teach the underlying capability honestly. The capability is the point, not the product. Every control in this guide exists in commercial form too, and Lab 28 is specifically about evaluating those on their merits. When a named tool has changed or is unavailable by the time you read this, substitute another that provides the same capability and the lab still works. The harden-detect-respond-manage spine does not depend on any one vendor.

End of guide.